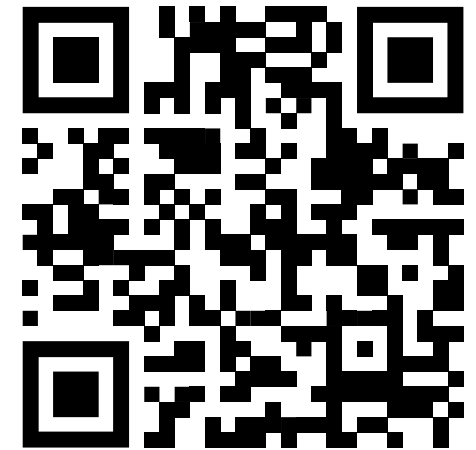


Programmierung in C++

Speicherverwaltung

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Vinzenz Funk



<https://poll.hs-kempten.de/poll/>

Speicherverwaltung

Einführung



- Der Programmierer erhält in C++ große Verantwortung über die Verwaltung des Prozessspeichers
 - Dies birgt viele Möglichkeiten zur **Performanzoptimierung** einer Anwendung,
 - aber auch viele **Risiken** die zu schwerwiegenden Laufzeitfehlern und Abstürzen eines Programms führen können
- Zur effizienten und sicheren Verwaltung des Speichers bedarf es ...
 - einem Grundverständnis der **Speicherbereiche** und Operationen auf diesen und ...
 - Methoden zur *manuellen* und *unterstützten* **Speicherverwaltung**
- Mit Einführung des **C++11** Standards wurde die Speicherverwaltung großflächig modernisiert und vereinheitlicht
 - Das Verständnis moderner Speicherverwaltung ist essenziell für die zukunftsfähige Entwicklung von C++-Anwendungen



Prozessspeicher

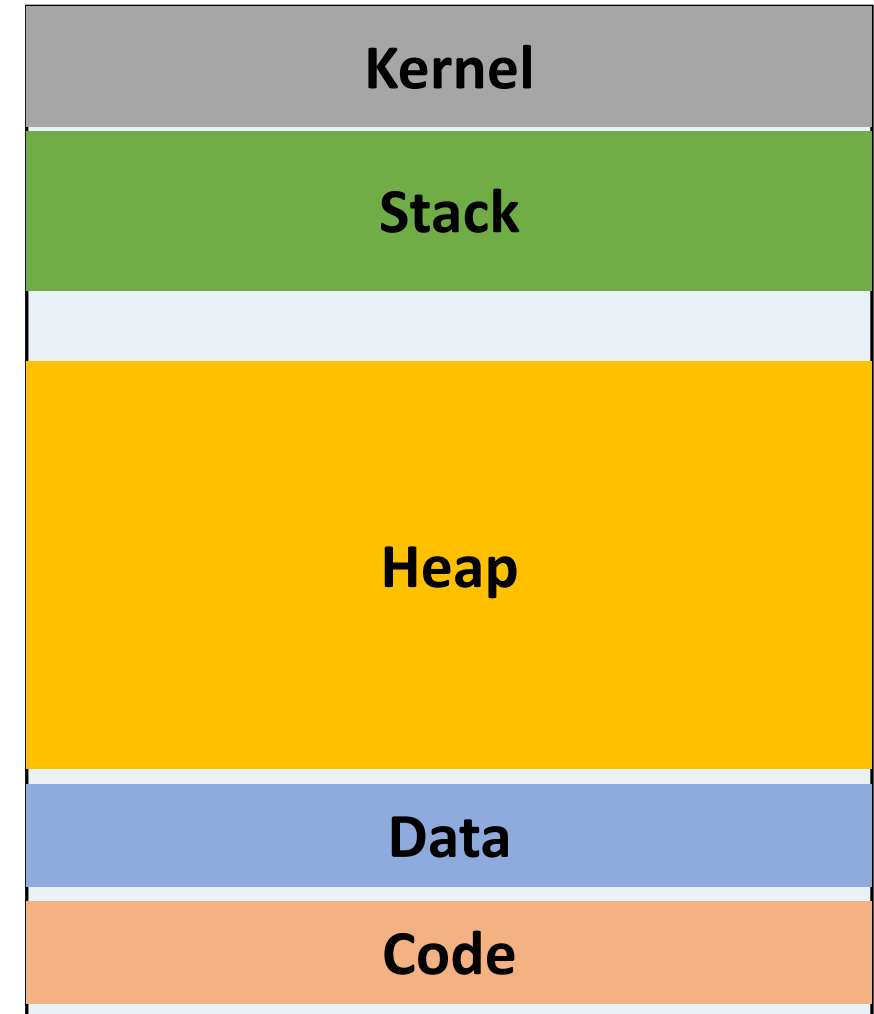
Lernziele

- Der **Aufbau** des Prozessspeichers und dessen **Teilbereiche** wurden verstanden
- Die **Auswirkung** von Operationen während der Programmausführung können auf den Prozessspeicher nachvollzogen werden

Prozessspeicher

Layout

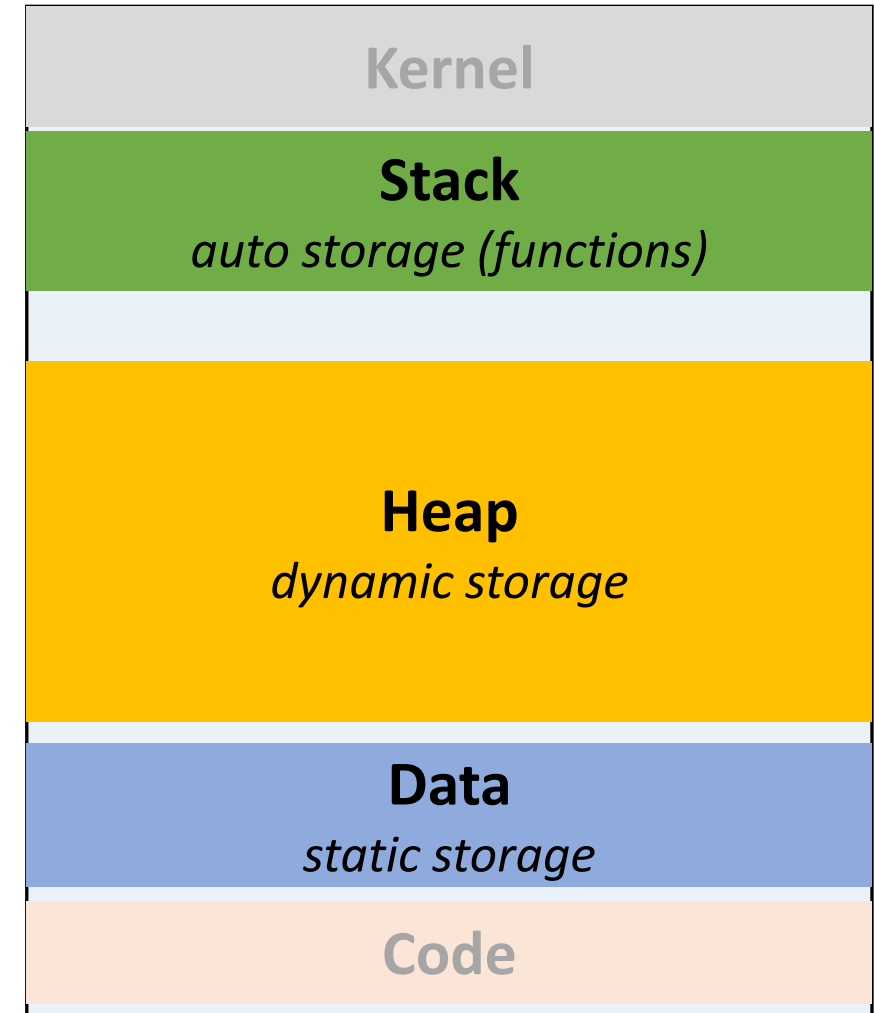
- Der **Prozessspeicher** ist in verschiedene Bereiche unterteilt
 - **Kernel**: Enthält Daten des Betriebssystems
 - **Stack**: Enthält Funktionen und deren lokale Variablen
 - **Heap**: Enthält dynamisch erzeugte Objekte und (meist große) Datenmengen
 - **Data**: Enthält globale und statische Variablen
 - **Code**: Enthält den Maschinencode (Von-Neumann-Architektur) – auch „Text“-Bereich genannt



Prozessspeicher

Layout

- Die Bereiche bilden die **Speicherdauer** von Objekten ab (Rückblick: Grundlagen)
 - **Automatische** Speicherdauer: Objekte liegen (bei Funktionen) im Stack-Bereich
 - **Dynamische** Speicherdauer: Objekte liegen im Heap-Bereich
 - **Statische** Speicherdauer: Objekte liegen im Data-Bereich

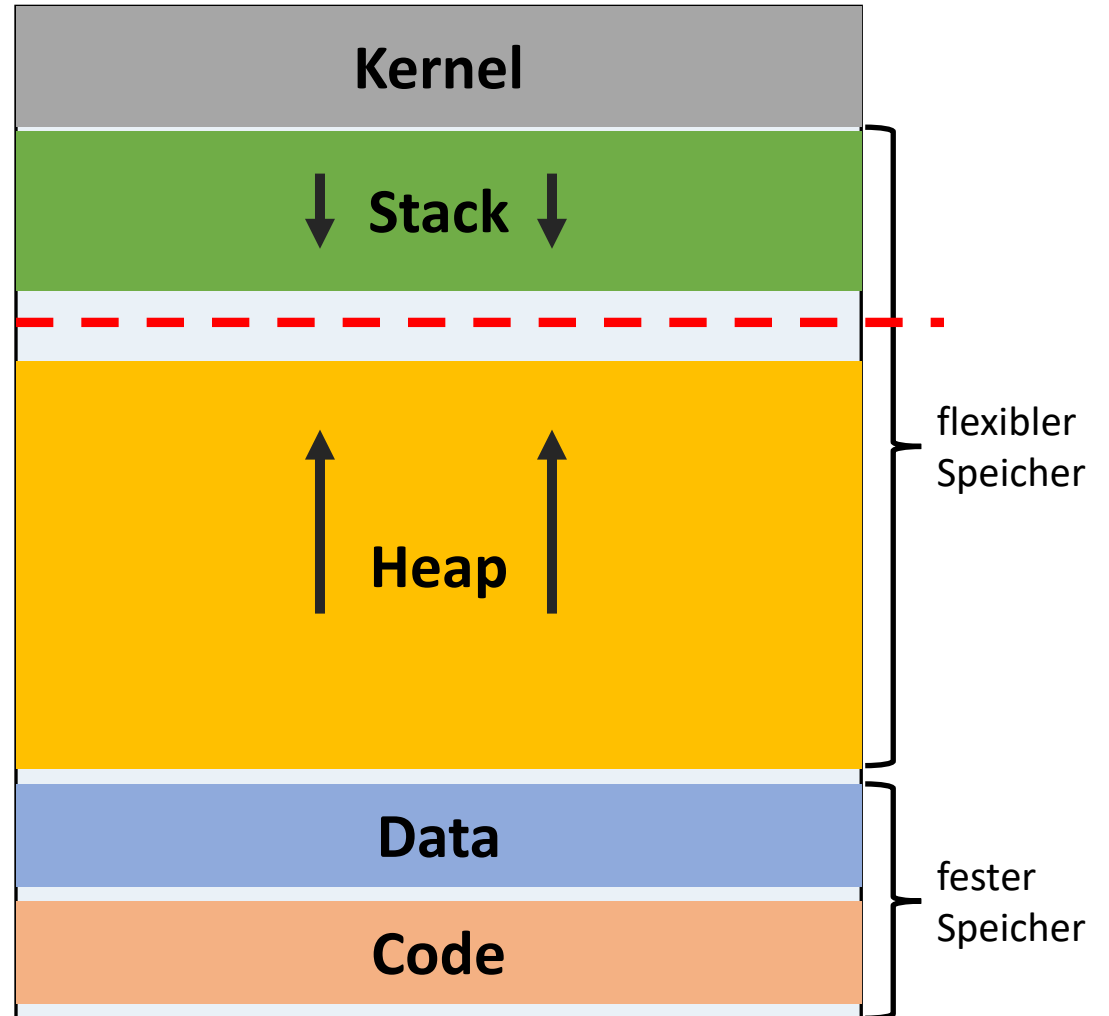


Prozessspeicher

Layout

- Auf den **Kernel**bereich besteht weder Schreib- noch Lese-Zugriff
 - Nur Funktionen der Kernelschicht (Betriebssystem) haben Zugriff
- Die **Stack**- und **Heap**-Bereiche skalieren dynamisch bis zu einer gewissen Grenze
 - Dabei ist der verfügbare Speicherbereich für den Stack deutlich geringer als für den Heap (1MB vs. mehrere GB in MSVC¹)
- Die **Data**- und **Code**-Bereiche werden bei der Kompilierung definiert und verändern ihre Größe während des Programmablaufs nicht

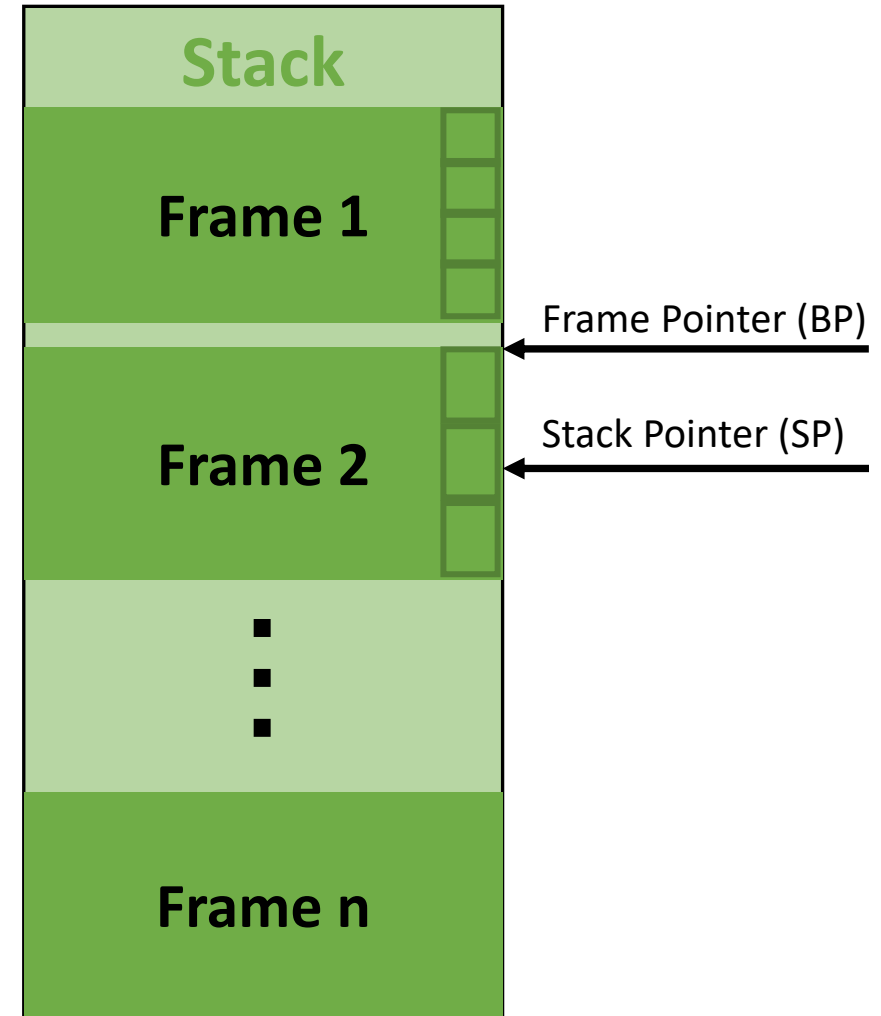
¹ Microsoft Visual C++ (Compiler)



Prozessspeicher

Stack

- Alle Daten einer Funktion werden als **Stackframe** abgebildet
- Beim Funktionsaufruf wird der Stackframe auf den Stack *gepusht* (Datenstruktur: Stack)
- Beim Funktionsende wird der Stackframe von dem Stack *gepoppt* (Datenstruktur: Stack)
- Zeiger auf die aktuellen Adressen im Stack befinden sich direkt in CPU-Registern (siehe LV „IT-Systeme“)
 - **BP** (Base Pointer): Zeigt auf den Anfang des aktuellen Stackframes
 - **SP** (Stack Pointer): Zeigt auf eine aktuell benutzte lokale Variable der Funktion





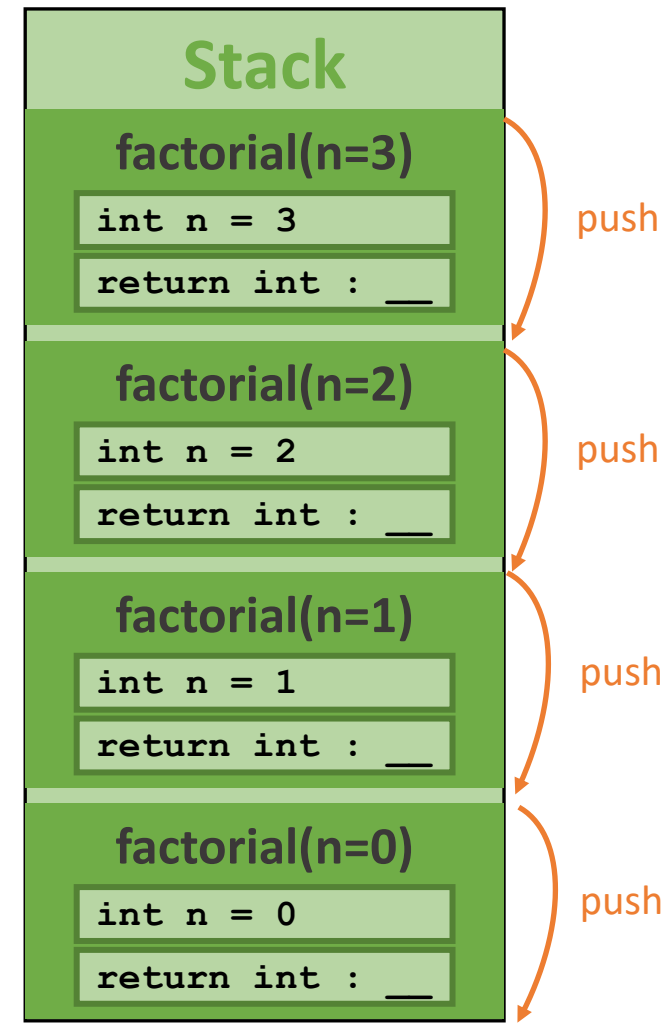
Prozessspeicher

Stack – Beispiel

- Beispiel: rekursive Fakultät von 3

```
// Fakultät von n
int factorial(int n)
{
    if (n == 0)
    {
        return 1;
    }
    else
    {
        // Rekursion
        return n * factorial(n - 1);
    }
}

int main()
{
    // 3!
    std::cout << factorial(3);
}
```



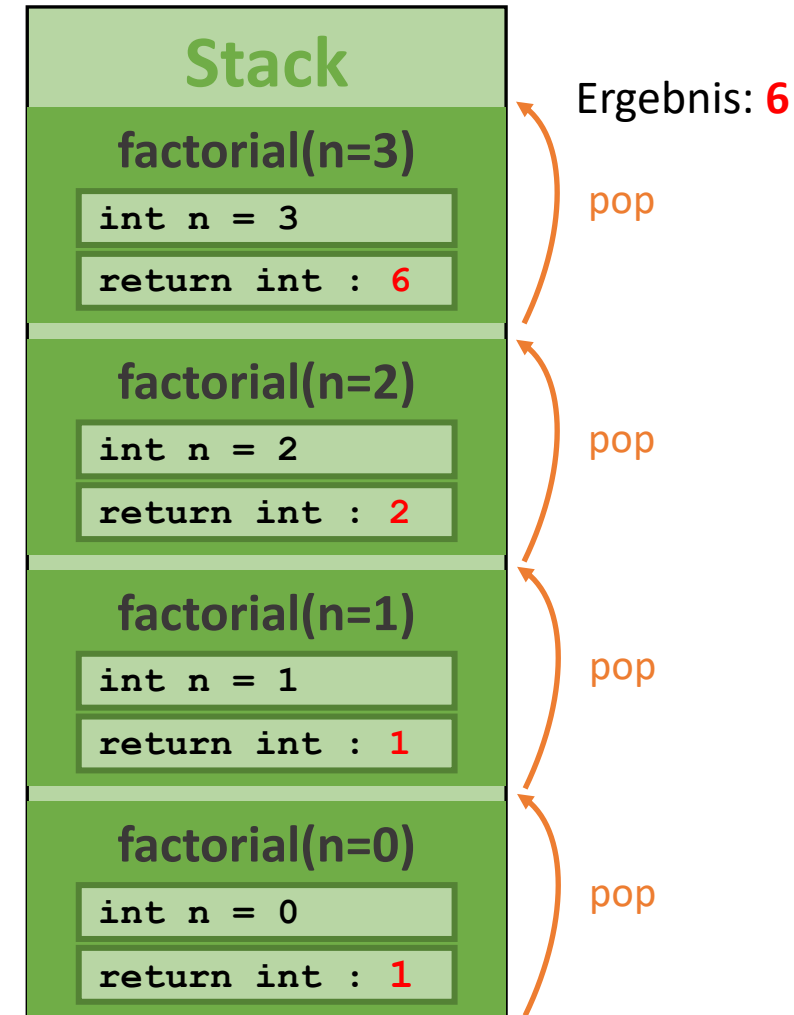
Prozessspeicher

Stack – Beispiel

- Beispiel: rekursive Fakultät von 3

```
// Fakultät von n
int factorial(int n)
{
    if (n == 0)
    {
        return 1;
    }
    else
    {
        // Rekursion
        return n * factorial(n - 1);
    }
}

int main()
{
    // 3!
    std::cout << factorial(3);
}
```



Prozessspeicher

Stack – Beispiel

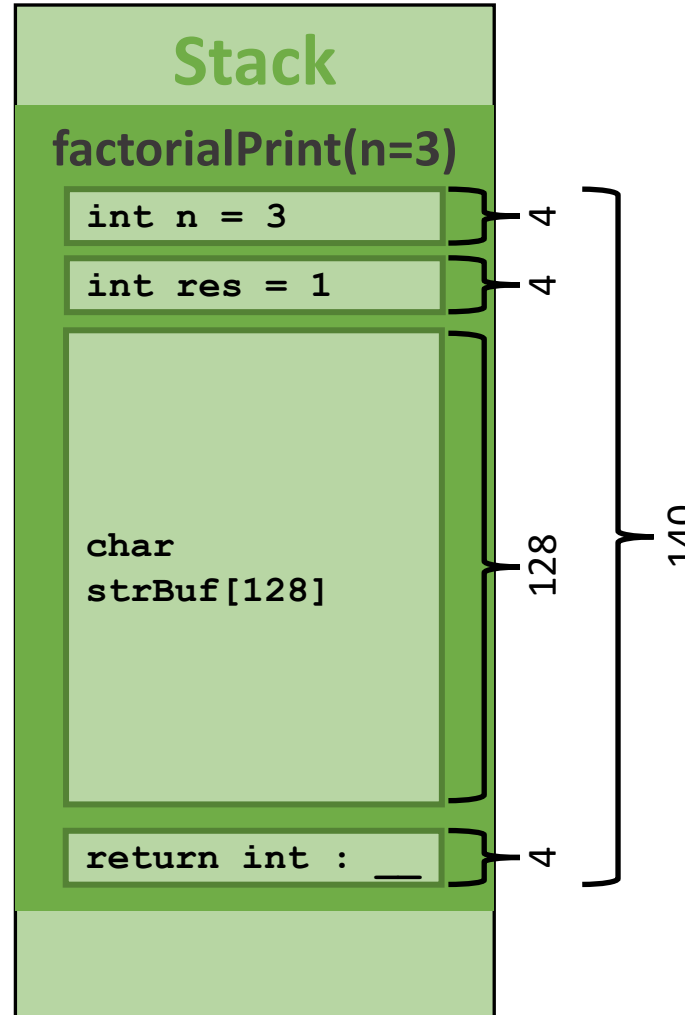
- Beispiel: großer Stackframe

```
int factorialPrint(int n)
{
    int res = 1;
    if (n == 0)
        return res;

    // Rekursion
    res = n * factorialPrint(n - 1);

    // Ausgabe
    char strBuf[128];
    sprintf(strBuf,
        "Factorial step %d: %d",
        n, res
    );
    puts(strBuf);

    return res;
}
```



- 140 Bytes
Speicherverbrauch
pro Funktionsaufruf
- Weniger Aufrufe
möglich als bei
`factorial()`

→ Große
Datenmengen sollten
nicht auf dem Stack
gespeichert werden



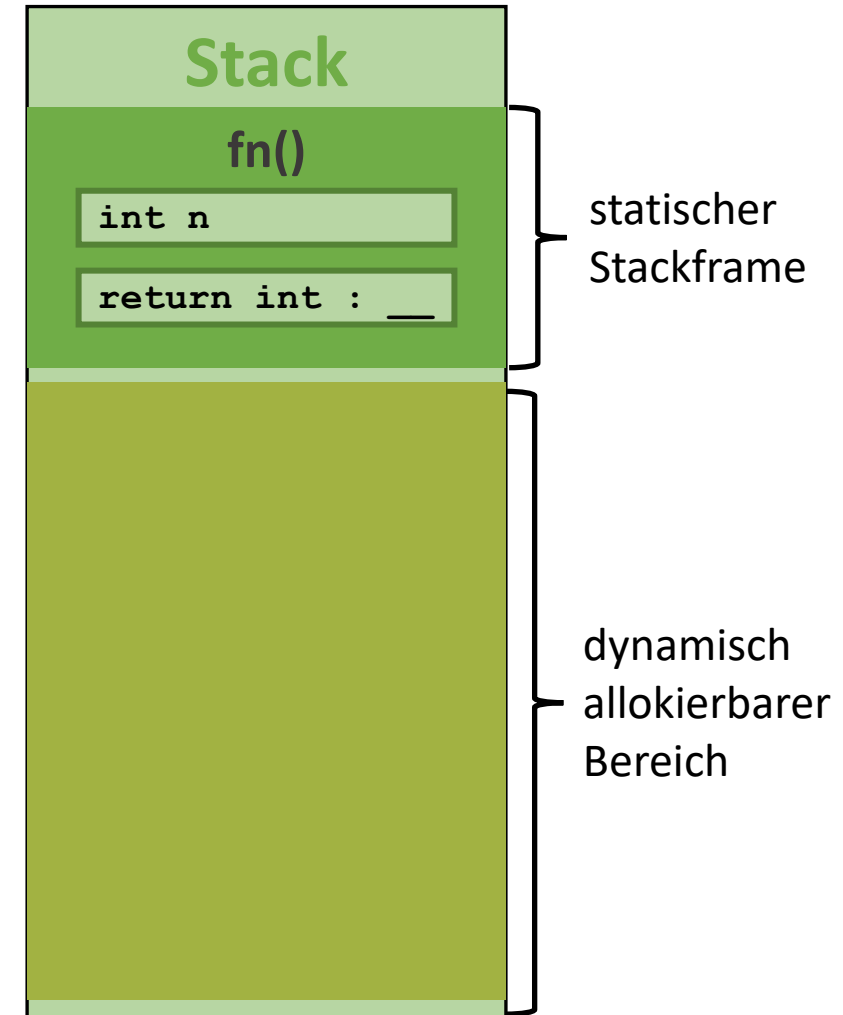


Prozessspeicher

Stack – Allokation

- Die Größe eines Stackframes ist zur Kompilierung bekannt und wird zur Laufzeit *statisch* allokiert
- Weiterer Stackspeicher kann *dynamisch* allokiert werden; z.B. mit der C-Funktion `alloca(n)`
 - Kein Äquivalent im C++-Standard
 - Nicht auf allen Systemen unterstützt
 - Vorteil: Performance

→ Stackallokationen sollten vermieden werden, da der Stackspeicher stark begrenzt und für Funktionsaufrufe reserviert ist



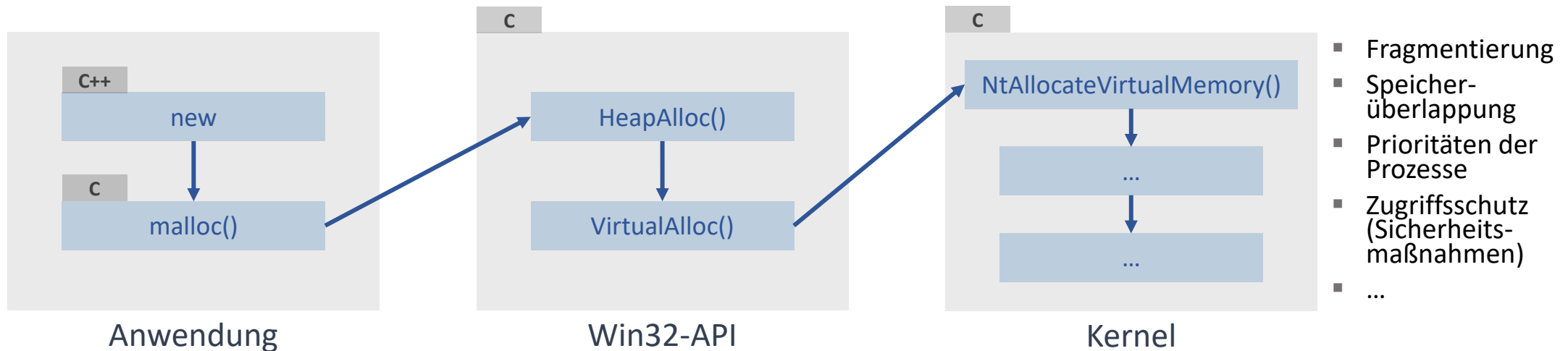
Prozessspeicher



Heap

- Der **Heap** ist der Speicherbereich, der einem Prozess für *dynamische* Allokationen zur Verfügung steht
- Die Allokationen werden vom Betriebssystem durchgeführt
 - Dabei wird physischer Speicher dem Prozess zugewiesen (virtueller Speicher)
- Allokationen beinhalten **zeitaufwändige Operationen** zur Speicherverwaltung (siehe LV „Betriebssysteme“)

Beispiel einer Heapallokation im „Windows“ Betriebssystem:



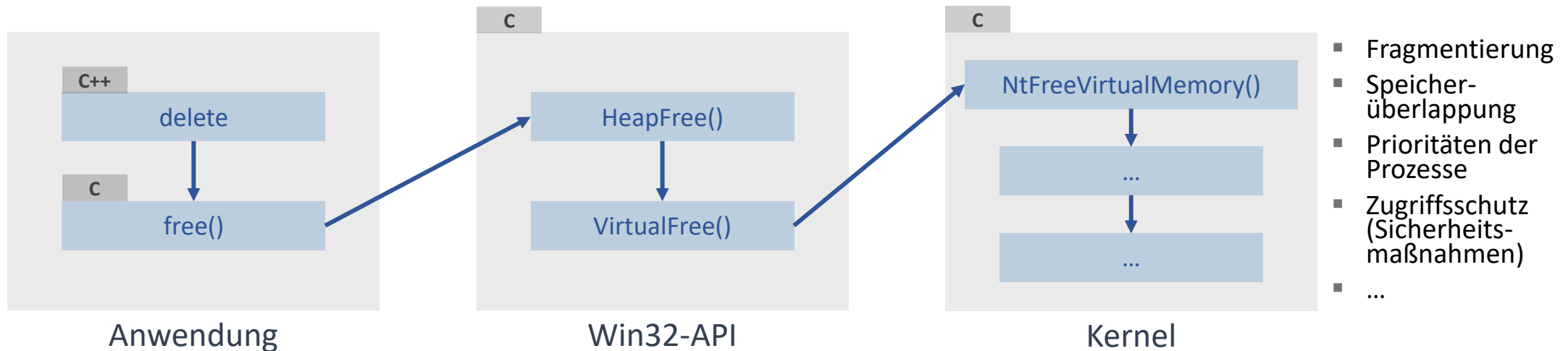
Prozessspeicher



Heap

- Aus dem Heap allozierter Speicher muss nach Verwendung wieder freigegeben werden (**Deallokation**, Freigabe)
 - Dabei wird der zuvor allokierte Speicherblock dem Betriebssystem zurückgegeben
 - Dieser wird u.U. später wiederverwendet
- Findet die Deallokation nicht statt, entstehen **Lecks** im Heap – sogenannte „**memory leaks**“ – die den Heap auslasten und letztendlich zum Absturz des Programms führen können

Beispiel einer Heapdeallokation im „Windows“ Betriebssystem:



Prozessspeicher



Heap – Beschädigungen

- Der Heap kann durch Programmierfehler beschädigt werden – dabei spricht man von einer „**heap corruption**“
- Häufige Fehlerquellen sind hierbei
 - **Buffer Overrun**: Es werden mehr Daten in einen allokierten Speicherbereich geschrieben, als in diesem zur Verfügung stehen (besonders häufiger Fehler bei Zeichenketten (strings))
 - **Use after Free**: Es wird auf Speicher zugegriffen, der bereits deallokiert wurde
 - **Double Free**: Ein allokiertes Speicherblock wird mehrmals deallokiert
- Beschädigungen am Heap führen immer zu „**undefined behavior**“ der Anwendung
 - Das Programm kann an einer beliebigen Stelle im Programmablauf abstürzen
 - Daten können verfälscht werden
 - Es kommt zu gravierenden Sicherheitsproblemen (exploits)
 - Das Programm kann problemlos weiterlaufen



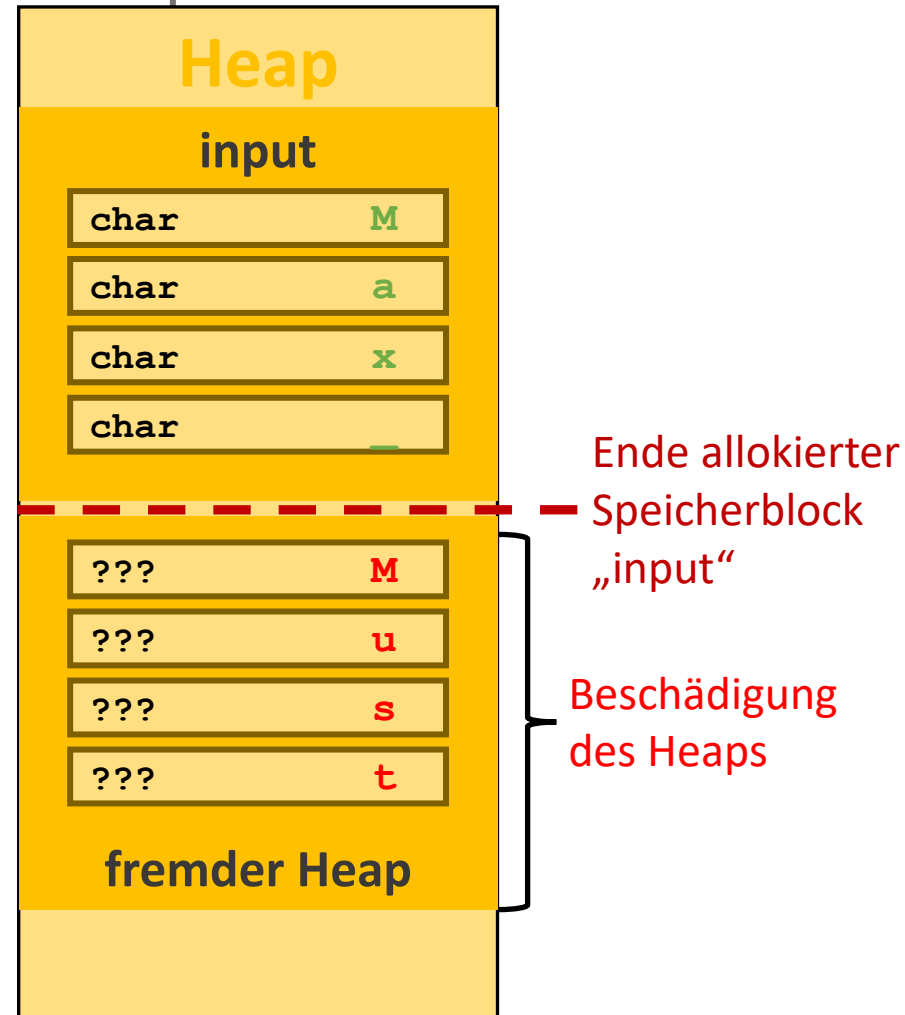
Prozessspeicher

Heap – Beschädigungen – Beispiel

■ Beispiel: **Buffer Overrun**

```
int main()
{
    char* input = new char[4];
    puts("Namen eingeben: ");
    gets(input);
}
```

Eingabe: **Max Mustermann**



- Eingabe überschreitet Grenze des allokierten Blocks
- Nachfolgender Block wird beschädigt
- Dieser könnte von anderem Programmteil in Verwendung sein

→ **undefined behavior**

Prozessspeicher

Heap – Beschädigungen –

■ Beispiel: Use after Free

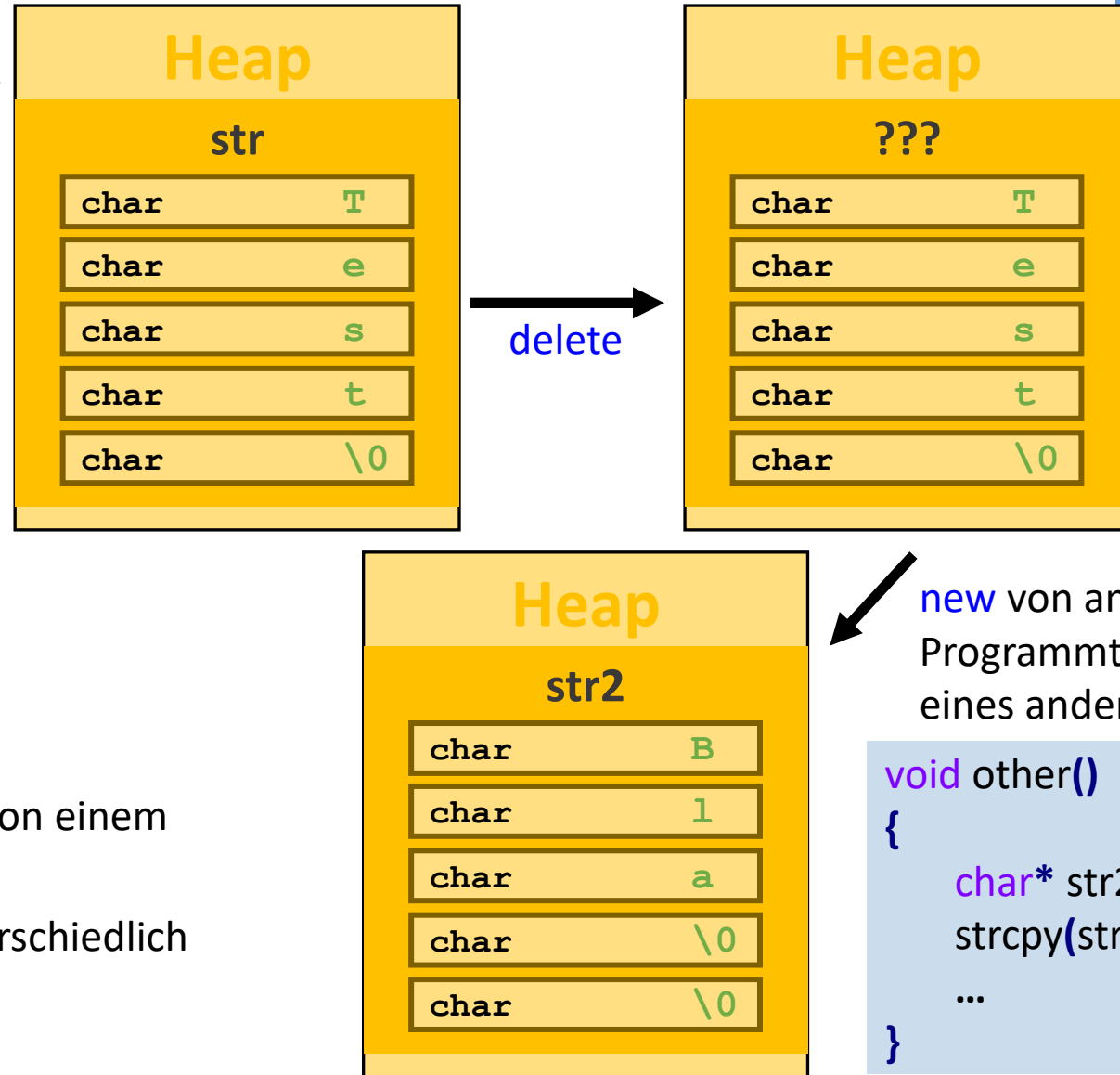
```
int main()
{
    char* str = new char[5];
    strcpy(str, "Test");
    delete[] str;

    std::cout << "Str is: " << str;
}
```

Str is: Bla

- Allokierter Block steht nach Deallokation einem anderen Programmteil zur Verfügung
- Gleicher Speicher wird mehrfach unterschiedlich verwendet

→ undefined behavior



new von anderem Programmteil und Setzen eines anderen Strings

```
void other()
{
    char* str2 = new char[4];
    strcpy(str2, "Bla");
    ...
}
```



Prozessspeicher

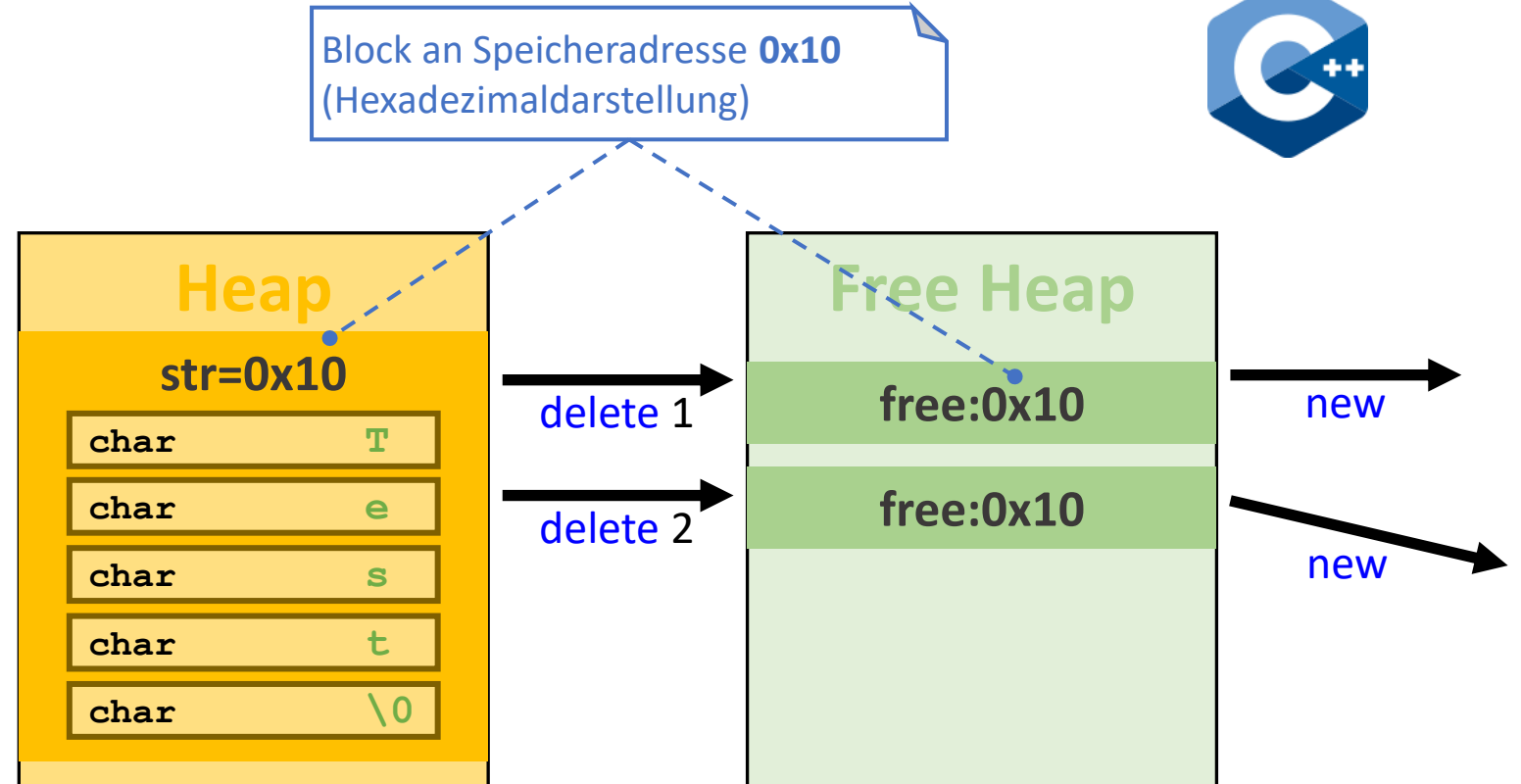
Heap

■ Beispiel: Double Free

```
int main()
{
    char* str = new char[5];
    strcpy(str, "Test");
    delete[] str;

    ...

    delete[] str;
}
```



- Der Block wird mehrmals als „wieder frei verfügbar“ markiert und u.U. mehrmals neu zugewiesen
- Dann entstehen ähnliche Probleme wie bei Use-after-free, wo der gleiche Speicherblock mehrfach verwendet wird

→ **undefined behavior**

Prozessspeicher

Zusammenfassung



- Der Prozessspeicher ist in fünf Bereiche unterteilt, von denen für den Programmierer vor allem der **Stack**- und der **Heap**bereich interessant sind
- Der **Stack** enthält Frames der aufgerufenen Funktionen, die deren lokale Variablen enthalten
 - Stackspeicher wird statisch für jeden **Funktionsaufruf** allokiert, kann aber auch dynamisch während eines Funktionsaufrufs allokiert werden
 - Stackspeicher ist **stark limitiert** und sollte nicht ausgedehnt werden
- Der **Heap** enthält dynamische – zum Teil auch sehr große – Allokationen
 - Allokationen werden vom Betriebssystem durchgeführt und sind **teuer**
 - Allokierter Heapspeicher muss wieder **deallokiert** werden
- Durch falsche Operationen auf dem Stack oder Heap können **schwerwiegende Speicherfehler** auftreten

Prozessspeicher

Verständnisfragen

PIN: 618987

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Im Stack-Bereich befinden sich lokale Daten eines Funktionsaufrufs.
- Der Stack- und der Heap-Bereich sind in etwa gleich groß.
- Der Kernel-Bereich kann nur vom Betriebssystem ausgelesen und beschrieben werden.
- Die Data- und Code-Bereiche ändern während der Programmausführung flexibel ihre Größe.
- Eine Heap-Allokation ist deutlich zeitaufwändiger im Vergleich zu einer Stack-Allokation.
- Eine Heap-Allokation wird durch den Kernel des Betriebssystems durchgeführt.
- Die Laufzeitumgebung schützt den Heap vor Beschädigungen durch den Programmierer.

■ Welches Problem kann durch diese Funktion auftreten?

```
void setText(const char* txt)
{
    char* buf = new char[32];
    strcpy(buf, txt);
}
```

- Stack-Beschädigung durch **Buffer Overrun**
- Heap-Beschädigung durch **Buffer Overrun**
- Heap-Beschädigung durch **Use after free**
- Heap-Beschädigung durch **Double Free**



Manuelle Speicherverwaltung

Lernziele

- Funktionen zur **manuellen Allokation** und **Deallokation** von Prozessspeicher wurden verstanden
- Die Einführung und Notwendigkeit der Operatoren **new** und **delete** in **C++** kann nachvollzogen werden

Manuelle Speicherverwaltung



Dynamischer Speicher

- Rückblick: Arten der Speicherdauer (*storage durations*)
 - **automatisch**: Das Objekt lebt so lange, wie es in seinem umschließenden Scope gültig ist
 - **dynamisch / manuell**: Das Objekt lebt so lange, bis es manuell durch den Programmierer gelöscht wird
 - **statisch**: Das Objekt lebt so lange, bis das gesamte Programm beendet wird
 - **temporär**: Das Objekt lebt so lange, bis es nicht mehr von Ausdrücken verwendet wird
 - **C++11 thread**: Das Objekt lebt so lange, bis der eigene Thread terminiert wird
- Zur dynamischen / manuellen Speicherverwaltung werden Funktionen benötigt, die Objekte explizit erzeugen und zerstören können
 - Bereits bekannt: **new/delete**-Operatoren
 - Diese Objekte werden im **Heap**-Bereich des Prozessspeichers allokiert und deallokiert
- Eine **Allokation** ist nur die Zuweisung des Speichers; Eine **Deallokation** die Freigabe des Speichers
 - Objekterzeugung und Zerstörung (Kon-/Destruktor) wird separat betrachtet

Manuelle Speicherverwaltung



malloc & free

- In der Programmiersprache **C** wird Heapspeicher dynamisch mit der Methode **malloc(n)** allokiert und mit der Methode **free(p)** deallokiert

<stdlib.h>	
<code>void* malloc(size_t size);</code>	• Allokiert einen Speicherblock der Größe „size“ in Bytes vom Heap • Gibt einen Pointer undefinierten Typs (<code>void*</code>) auf das erste Byte in diesem allokierten Block zurück • Schlägt die Allokation fehl, wird stattdessen ein Nullpointer zurückgegeben
<code>void free(void* ptr);</code>	• Gibt einen, durch malloc allokierten, Speicherblock wieder frei

- Mit malloc allokiert Speicher wird *nicht initialisiert* – in ihm befindet sich **Speichermüll** (*memory garbage*)
 - Mit der Funktion **calloc(n, s)** kann Speicher allokiert und auf Nullen initialisiert werden
- Ein mit malloc allokiert Block kann mit der Funktion **realloc(p, s)** vergrößert und verkleinert werden, wenn das Betriebssystem dies unterstützt
- Diese Art der Allokation und Deallokation ist auch in C++ verfügbar

Manuelle Speicherverwaltung

malloc & free – Beispiel



- Beispiel: Allokation einer Zeichenkette

```
int main()
{
    // Allokation einer Zeichenkette (mit 256 Zeichen)
    // malloc() gibt void* zurück
    // → explizierter Cast notwendig
    char* input { static_cast<char*>(
        malloc(256)
    )};

    puts("Namen eingeben: ");
    gets(input);
    processInput(input);

    // Deallokation der Zeichenkette
    free(input);
}
```

- Beispiel: Allokation einer Klasseninstanz

```
int main()
{
    // Allokation einer "Oger" Instanz
    // Größe des benötigten Speichers muss manuell festgestellt werden
    Oger* oger { static_cast<Oger*>(
        malloc(sizeof(Oger))
    )};

    oger->angreifen();

    // Deallokation der Instanz
    free(oger);
}
```

...hier fehlt doch etwas?

Manuelle Speicherverwaltung

malloc & free – Probleme



- Die manuelle Speicherallokation mit malloc() beinhaltet **Unschönheiten** und **große Risiken**
 - Die **Größe** des zu-allokierenden Speichers muss manuell (z.B. mit der **sizeof()**-Funktion) festgestellt werden
 - mehr Schreibaufwand
 - hohes Fehlerpotential durch falsche Berechnungen
 - Da der **Rückgabewert** von malloc() ein Void-Pointer ist, muss dieser fast immer explizit gecastet werden, bevor der Speicher verwendet werden kann
 - mehr Schreibaufwand
 - hohes Fehlerpotential durch falsche Casts
 - **Konstruktoren** und **Destruktoren** von Klasseninstanzen werden nicht implizit aufgerufen
 - manueller Aufruf nötig
 - unschön und hohes Fehlerpotential bei Vergessen
 - Bei **Fehlern** während der Allokation wird lediglich ein Nullpointer zurückgegeben
 - leichtes Übersehen von Fehlern
 - Absturz durch fehlende Fehlerbehandlung

Manuelle Speicherverwaltung



malloc & free – Probleme

- Die manuelle Speicherdeallokation mit free() beinhaltet ebenfalls **große Risiken**
 - free() muss **explizit aufgerufen** werden, um Speicher wieder freizugeben
→ memory leaks bei Vergessen
 - free() kann **mehrmals für den gleichen Speicherblock** aufgerufen werden
→ double free
→ heap corruption
 - Pointer auf allokierten Speicher können auch **nach dem Aufruf** von free() **verwendet** werden
→ use after free
→ heap corruption

Manuelle Speicherverwaltung



new & delete

- In **C++** sind die *Operatoren* **new** und **delete** für dynamische Heapallokationen definiert

C++ Expressions	
new type	• Allokiert und initialisiert ein Objekt des angegebenen Typs (type) • Bei dem Typ kann es sich auch um einen einfachen Datentyp handeln (int, float, ...) • Erlaubt die Angabe von Konstruktorparametern (new-initializer) • Gibt einen Pointer des Objekttyps (type*) auf das erstellte Objekt zurück • Schlägt die Erstellung fehl, wird eine Exception ausgelöst
new type new-initializer	
delete ptr	• Zerstört und deallokiert ein Objekt, das zuvor mit new erstellt wurde

(Mit **new** kann ein Objekt auch ohne Allokation initialisiert werden – d.h. auf bereits-vorhandenem Speicher. Diese Operation nennt man „Placement-New“)

Dies führt über den Rahmen der Veranstaltung hinaus und wird nicht näher betrachtet.

Manuelle Speicherverwaltung



new & delete

- Die Operatoren **new** und **delete** sind **objektorientiert**
 - **new** wird direkt mit dem zu-allokierenden Objekttyp aufgerufen, statt der Bytegröße
→ keine **sizeof**-Operationen nötig
 - **new** gibt einen typisierten Pointer auf das allokierte Objekt zurück
→ keine Notwendigkeit für explizite Casts
 - **new** wirft im Fehlerfall eine C++-Exception (Erklärung folgt in VL „Fehlerbehandlung“)
→ kein Return von Nullpointern
 - **new** ruft den **Konstruktor** und **delete** den **Destruktor** einer Klasseninstanz (= Objekt) implizit auf
→ kein manueller Aufruf nötig
 - Allerdings wird auch beim new-Operator allozierter Speicher **nicht initialisiert**, sofern der Konstruktor das nicht tut!
- Eine Vergrößerung allokierten Speichers, wie durch die **realloc()**-Funktion, ist nicht möglich

Manuelle Speicherverwaltung



new & delete – Beispiel

- Beispiel: Klasseninstanz mit new und delete

```
int main()
{
    // Allokation einer "Oger" Instanz
    // Ruft direkt den Konstruktor auf
    Oger* oger {
        new Oger("Grump", 10, 1)
    }

    oger->angreifen();

    // Deallokation der Instanz
    // Ruft zuerst den Destruktor auf
    delete oger;
}
```

Mit **malloc()** und **free()**
sähe diese Operation so aus:

```
int main()
{
    // Allokation einer "Oger" Instanz
    Oger* oger { static_cast<Oger*>(
        malloc(sizeof(Oger))
    )};
    // Konstruktor-Aufruf
    oger->Oger("Grump", 10, 1);

    oger->angreifen();

    // Destruktor-Aufruf
    oger->~Oger();
    // Deallokation der Instanz
    free(ogер);
}
```

Manuelle Speicherverwaltung



new & delete – Beispiel

- Viele Implementierungen der new- und delete-Operatoren greifen intern auf **malloc()** und **free()** zurück

```
template <typename T>
T* newFn()
{
    // Allokation und Fehlerbehandlung
    void* pBlock { malloc(sizeof(T)) };
    if (pBlock == nullptr)
        throw std::bad_alloc;

    // Cast in Zieltyp
    T* pObj { static_cast<T*>(pBlock) };

    // Konstruktor-Aufruf
    pObj->T(...);

    return pObj;
}
```

```
template <typename T>
void deleteFn(T* pObj)
{
    // Destruktor-Aufruf
    pObj->~T();

    // Deallokation
    free(pObj);
}
```

→ Besser, jedoch nicht alle
Probleme beseitigt
(Erklärung folgt)

Manuelle Speicherverwaltung



new[] & delete[]

- Für Allokationen von **Arrays** sind weiter die *Operatoren* **new[n]** und **delete[]** definiert

C++ Expressions	
new type [size]	• Allokiert und initialisiert mehrere Objekte des angegebenen Typs (type) • Die Anzahl der zu-erstellenden Objekte wird in eckigen Klammern angegeben ([size]) • Bei dem Typ kann es sich auch um einen einfachen Datentyp handeln (int, float, ...) • Erlaubt keine Angabe von Konstruktorparametern • Gibt einen Pointer des Objekttyps (type*) auf das erste erstellte Objekt zurück • Schlägt die Erstellung fehl, wird eine Exception ausgelöst
delete []	• Zerstört und deallokiert alle Objekte, die zuvor mit new[] erstellt wurden • Eine Angabe der Anzahl erfolgt nicht → Klammern bleiben leer

- Der **new[]**-Operator ruft die Konstruktoren und der **delete[]**-Operator die Destruktoren der einzelnen Objekte implizit auf
 - Es können jedoch keine Parameter für die Konstruktoren übergeben werden
→ Nur Objekte mit einem **Standardkonstruktor** können mit dem new[]-Operator erstellt werden

Manuelle Speicherverwaltung



`new[]` & `delete[]`

- Mit **`new[n]`** allokierte Arrays müssen mit **`delete[]`** wieder freigegeben werden, **nicht mit dem `delete`-Operator (ohne Klammern)!**
- Warum **`delete[]`**?
 - Beim „normalen“ **`delete`**-Operator wird nur ein Destruktor, nämlich der des ersten Objekts im Array, aufgerufen
 - Der **`delete[]`**-Operator ruft die Destrukturen aller Objekte im Array auf, bevor der Speicherblock freigegeben wird
 - Neben der Allokation speichert der Compiler beim Aufruf des **`new[]`**-Operators dafür häufig auch die Anzahl der allokierten Objekte im Speicherblock mit ab
 - Dieser zusätzliche Speicher wird ebenfalls nicht korrekt deallokiert, wenn nicht der **`delete[]`**-Operator aufgerufen wird
- Der Aufruf des falschen **`delete`**-Operators kann bei trivialen Objekten „gut gehen“, bei komplexeren Destrukturen aber zu großen Problemen führen
→ **undefined behavior**

Manuelle Speicherverwaltung



new[] & delete[] – Beispiel

- Beispiel: Allokation einer Zeichenkette

```
int main()
{
    // Allokation einer Zeichenkette
    // new[] gibt jetzt char* zurück
    // => kein explizierter Cast notwendig
    char* input { new char[256] };

    puts("Namen eingeben: ");
    gets(input);
    processInput(input);

    // Deallokation der Zeichenkette
    delete[] input;
}
```

- Beispiel: Allokation eines Arrays von 4 Integern

```
int main()
{
    // Allokation 4 Elemente
    // keine sizeof()-Operation nötig
    int* ints { new int[4] };

    ints[0] = 2;
    ints[1] = 3;
    ints[2] = 5;
    ints[3] = 8;

    // Deallokation des Arrays
    delete[] ints;
}
```

Manuelle Speicherverwaltung



new & delete – Probleme

- Die **new**- und **new []**-Operatoren lösen die Probleme der **malloc()**-Funktion
 - Keine **sizeof()**-Aufrufe mehr nötig
 - Kein explizites Casting des Rückgabewerts
 - Konstruktoren und Destruktoren von Klasseninstanzen werden implizit aufgerufen
 - Fehler während Allokation und Initialisierung führen zu Exceptions
- Probleme der manuellen Speicher**deallokation** bleiben jedoch bestehen
 - **delete** bzw. **delete []** muss explizit aufgerufen werden
 - **delete** bzw. **delete []** kann mehrmals für den gleichen Speicherblock aufgerufen werden
 - Zeiger auf allokierten Speicher können auch nach dem Aufruf von **delete** bzw. **delete []** verwendet werden
- Hinzu kommt die neue Problematik, dass für Allokationen mit dem **new []**-Operator der falsche **delete**-Operator aufgerufen werden kann (und umgekehrt **delete []** für **new**)
 - Auch darf **malloc()** und **free()** nicht mit **new** und **delete** vermischt werden

Manuelle Speicherverwaltung



Zusammenfassung

- Bei der manuellen Speicherverwaltung werden **einzelne Funktionen** zur Allokation oder Deallokation von Heap-Prozessspeicher aufgerufen
- Es sind zwei unterschiedliche Arten von speicherverwaltenden Funktionen verfügbar
 - **malloc() & free()**
Operation auf Binärdaten, aus C
 - **new, new[], delete, delete[]**
Operation auf Objekten, neu in C++
- Durch die **new**- und **delete**-Operatoren werden viele Probleme der **malloc()** - und **free()** -Funktionen vermieden
→ **new** und **delete** sollten daher bevorzugt eingesetzt werden
- Bei beiden Arten der manuellen Speicherverwaltung bleiben **Probleme** durch Vergessen oder falschen Aufrufs der **deallozierenden Funktionen** bestehen

Manuelle Speicherverwaltung

Verständnisfragen

PIN: 618987

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Eine Speicherallokation ist die Zuweisung von Arbeitsspeicher an einen Prozess.
- Eine Speicherallokation beinhaltet den Konstruktoraufruf eines Objekts.
- Die **malloc()** -Funktion gibt einen Zeiger auf den zu-allocierenden Typen zurück.
- Mit der **malloc()** -Funktion allocierter Speicher kann vergrößert und verkleinert werden.
- Mit dem **new**-Operator allocierter Speicher wird automatisch auf 0en initialisiert.
- Mit dem **new**-Operator allocierter Speicher kann problemlos mit der **free()**-Funktion freigegeben werden.
- Objekte ohne Standardkonstruktor können mit dem **new[]** -Operator erzeugt werden.
- Der **delete[]** -Operator ruft die Destruktoren aller Objekte eines Arrays auf.



RAII

Lernziele

- Die Idee der **unterstützten Speicher- und Ressourcenverwaltung** wurde anhand des modernen Ansatzes „RAII“ verstanden
- Das Konzept „**RAII**“ kann für eigene Objekte angewendet werden
 - Dazu wird in diesem Kapitel die Klasse „Oger“ aus der Vorlesung „Vererbung“ beispielhaft mit einem RAII-Objekt aufbereitet

RAII

Destruktoraufrufe

- Der Destruktor eines Objekts wird aufgerufen wenn das Objekt zerstört wird; Dies ist der Fall bei:
 1. **Ende eines Gültigkeitsbereichs** (scope)
Bei Objekten mit automatischer Speicherdauer
Z.B. Ende einer Funktion
 2. **Expliziter Zerstörung des Objekts**
Bei Objekten mit dynamischer Speicherdauer
Z.B. mit dem **delete**-Operator
 3. **Zerstörung eines verknüpften Objekts**
Z.B. Elternobjekt einer Hierarchie, Containerobjekt, Smart-Pointer, andere Datenstrukturen, ...
- Die impliziten Aufrufe von Destrukturen in C++ bilden die Basis für RAII



1. Ende Gültigkeitsbereich (scope)

```
int main()
{
    Oger oger("Grump", 10, 1);
    ...
} // Ende Scope von main() → Destruktor
```

2. Explizite Zerstörung

```
int main()
{
    Oger* oger { new Oger("Grump", 10, 1) };
    ...
    delete oger; // Explizite Zerstörung → Destruktor
}
```

3. Verknüpftes Objekt

```
struct Ogers { Oger o1, Oger o2 };
int main()
{
    Ogers ogers {
        Oger("Grump", 10, 1),
        Oger("Fergus", 2, 1)
    };
} // Verknüpfte Zerstörung → Destruktor
```

RAII



Programmiertechnik

- „**Resource Acquisition Is Initialization**“ (kurz **RAII**) ist eine Programmiertechnik, bei der **(Hilfs-)Klassen** zur Verwaltung von Ressourcen – wie Allokationen, Dateien, Threads, ... – erstellt werden, statt diese direkt zu verwalten
- Die Grundidee hierbei ist, dass...
 - der **Konstruktor** eine Ressource **akquiriert** (acquisition)
 - der **Destruktor** eine Ressource wieder **freigibt**
- Die Ressource darf von keinen anderen Methoden – wie `init()` oder `finalize()` – akquiriert oder freigegeben werden
- Von größerem Nutzen als die Akquirierung bei der Konstruktion ist die **gesicherte Freigabe** der Ressource bei der Objektzerstörung
 - Name „RAII“ ist nicht optimal gewählt

RAII

Beispiel



- Erinnern wir uns an das Beispiel der Allokation einer „Oger“-Klasseninstanz:

```
int main()
{
    // Allokation einer "Oger" Instanz
    Oger* oger { new Oger("Grump", 10, 1) };

    oger->angreifen();

    // Deallokation der Instanz
    delete oger;
}
```

- Um den Problemen der manuellen Speicherdeallokation Abhilfe zu schaffen, können wir eine RAII-Hilfsklasse **OgerPointer** definieren:

RAII

Beispiel



```
class OgerPointer
{
public:
    OgerPointer(const std::string& name, int hp, int hpClub)
    {
        m_pointer = new Oger(name, hp, hpClub);
    }

    ~OgerPointer()
    {
        delete m_pointer;
    }

    Oger* get() const
    {
        return m_pointer;
    }

private:
    Oger* m_pointer;
};
```

Akquirierung (acquisition) der Allokation im Konstruktor

Freigabe der Allokation im Destruktor

Zugriff auf die Ressource von außerhalb der Hilfsklasse. Der Rückgabewert kann auch eine Referenz sein

„Ummantelter“ Pointer auf die Allokation. Wichtig: **private**

RAII

Beispiel



- Mit dieser RAII-Hilfsklasse können wir auf die manuelle Speicherdeallokation verzichten

```
int main()
{
    // Erzeugung
    OgerPointer oger("Grump", 10, 1);

    // Nutzung
    oger.get()->angreifen();
}
```

Konstruktor allokiert Oger-Instanz und gibt die Parameter weiter

Die ummantelte Instanz kann über die Getter-Funktion genutzt werden

1. Der **Gültigkeitsbereich** der main()-Funktion endet
2. Das führt zur **Zerstörung** der **lokalen Variable** „oger“
3. Dies löst den **Destruktoraufruf** unserer RAII-Hilfsklasse „OgerPointer“ auf
4. Dieser führt die **Deallokation** der Oger-Instanz aus

→ Aufruf der Deallokation wurde gesichert

RAII

Beispiel



- Der Aufruf des Destruktors – und damit die Deallokation – ist auch gewährleistet wenn die Funktion frühzeitig verlassen wird

```
int main()
{
    OgerPointer oger("Grump", 10, 1);

    if (...) return;

    if (...) throw std::runtime_error{...};

    oger.get()->angreifen();
}
```

Frühzeitiges Verlassen durch **return**
→ Ruft Destruktor auf

Frühzeitiges Verlassen durch **throw**
→ Ruft Destruktor auf

Reguläres Verlassen durch **Scope-Ende**
→ Ruft Destruktor auf



- RAII-Klassen sollten **leichtgewichtig** sein und sich nur um die Verwaltung **einer einzelnen** Ressource kümmern
 - Komplexe Klassen sollten mehrere kleine RAII-Objekte nutzen, statt diese selbst zu verwalten
- RAII-Konstrukte stellen **keine** Form von **garbage collection** dar!
 - Der Aufruf des Destruktors ist immer **deterministisch**, wenn auch manchmal nicht gut sichtbar
→ Kein Finalisierer-Problem (= Garbagecollector nicht beeinflussbar) wie in Java, C#, Python, ...
- RAII kann vielseitig anderweitig angewendet werden, z.B.
 - Warten auf einen Thread, wenn eine Funktion verlassen wurde
 - Ausgabe einer Log-Meldung, wenn eine Funktion verlassen wurde
 - ...

RAII



Zusammenfassung

- Mit der RAII-Programmiertechnik können einzelne Ressourcen – wie Speicherallokationen – **ummantelt** (wrapping) werden
- Es werden **Aktionen zur Objektzerstörung** über einen Destruktor definiert, der mit der ummantelten Ressource fest verknüpft wird
- Der Hauptnutzen von RAII besteht in der **Absicherung** des Aufrufs von **Deallokationen** oder anderer **Ressourcefreigaben** bei Objektzerstörung
 - Statt dem manuellen Aufruf der Funktionen, wird vom impliziten Aufruf der Destrukturen Gebrauch gemacht



■ **Welche der folgenden Erklärungen beschreibt das Prinzip von RAII am besten?**

- Ein Mechanismus, der die erfolgreiche Objekterstellung sicherstellt.
- Ein Muster, das die Operatoren **new** und **delete** erweitert.
- Eine Technik, bei der eine Ressource ummantelt wird und die deren korrekte Freigabe sicherstellt.
- Ein Ansatz, der sicherstellt, dass Objekte nicht vor Programmende zerstört werden.

■ **WAHR oder FALSCH?**

- Durch die Programmier Technik "RAII" können memory leaks verhindert werden.
- Bei einer RAII-Klasse darf eine Ressource nur vom Konstruktor erstellt und vom Destruktor zerstört werden.
- Bei einer RAII-Klasse ist die Freigabe der Ressource auch sichergestellt, wenn eine Funktion frühzeitig verlassen wird.
- Bei einer RAII-Klasse kann nicht mehr festgestellt werden, wann genau die Freigabe der Ressource stattfindet.



Smart Pointer

Lernziele

- Die Abstraktion der Speicherverwaltung über **Eigentumsrechte** (object ownership) wurde verstanden
- Die **Smart-Pointer**-Klassen der Standardbibliothek können in eigenen Projekten korrekt eingesetzt werden

Smart Pointer



- Um den Problemen der manuellen Speicherverwaltung Abhilfe zu schaffen, definiert der **C++11**-Standard sogenannte „**Smart Pointer**“
- Dabei handelt es sich um **RAII-Klassen**, die intern einen Pointer und die damit verknüpfte Speicherallokation ummanteln
- Smart-Pointer übernehmen die Aufrufe der **new**- und **delete**-Operatoren und bieten dem Programmierer eine **besser nutzbare Abstraktion** zur dynamischen Speicherverwaltung
- Smart-Pointer sind in der **<memory>**-Bibliothek der Standardbibliotheken definiert
- Der Typ des zu ummantelnden Objekts wird als **Template-Argument** spezifiziert (*Erläuterung zu Templates folgt in kommender Vorlesung*)

Smart Pointer



Eigentumsrechte (object ownership)

- Smart-Pointer führen ein neues Konzept ein: Das **Eigentumsrecht (*ownership*)** an einem Objekt
- Ein Pointer, dem ein Objekt gehört, ist für das **Löschen des Objekts** nach dessen Nutzung verantwortlich (***strong reference***)
- Ein Pointer, der lediglich auf ein Objekt verweist, hat keinen Einfluss auf die Lebenszeit des Objekts (***weak reference***)
 - Ein Pointer, der auf ein gelöscht Objekt zeigt, nennt man auch „**dangling pointer**“ („baumelnder Zeiger“)
- Der Standard definiert drei Arten von Smart-Pointern, die sich in ihrem **Eigentumsverhalten** unterscheiden:

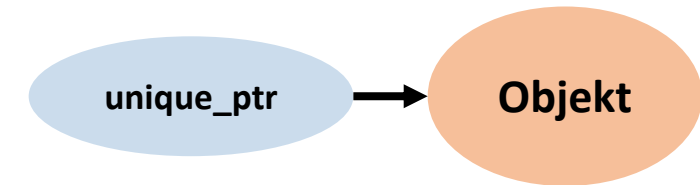
<memory>	
<code>unique_ptr<T></code>	für den <i>einmaligen</i> Besitz eines Objekts
<code>shared_ptr<T></code>	für den <i>gemeinschaftlichen</i> Besitz eines Objekts (unter mehreren Pointern)
<code>weak_ptr<T></code>	für <i>Verweise</i> auf ein Objekt, das einem <code>shared_ptr</code> gehört (auch <i>dangling</i>)

Smart Pointer

unique_ptr



- **unique_ptr** stellen den **einmaligen Besitz** eines Objekts dar
 - Das Objekt, das der `unique_ptr` *besitzt*, kann von keinem anderen Pointer geteilt oder referenziert werden
- Wichtigste Funktionen der Klasse:



<code>unique_ptr<T></code>	
<code>unique_ptr()</code>	Initialisiert einen leeren <code>unique_ptr</code>
<code>unique_ptr(T*)</code>	Initialisiert einen <code>unique_ptr</code> , der den Besitz des übergebenen Objekts übernimmt
<code>~unique_ptr()</code>	Zerstört das besessene Objekt (sofern eines vorhanden ist)
<code>T* get()</code>	Gibt einen Pointer auf das besessene Objekt zurück
<code>T* operator->()</code>	
<code>T& operator*()</code>	Gibt eine L-Value-Reference auf das besessene Objekt zurück
<code>operator bool()</code>	Gibt zurück ob ein Objekt besessen wird (<code>true</code>) oder nicht (<code>false</code>)
<code>T* release()</code>	Gibt den Besitz des Objekts ab und gibt den Pointer darauf zurück
<code>void reset(T* = nullptr)</code>	Ersetzt das besessene Objekt mit einem anderen. Das zuvor besessene Objekt wird zerstört

`unique_ptr` unterstützen auch die Angabe einer eigenen Löschfunktion für das Objekt – sogenannte „**deleter**“.
Dies führt über den Rahmen der Veranstaltung hinaus und wird nicht näher betrachtet. Siehe dazu Meyers „Effective Modern C++“

Smart Pointer



unique_ptr

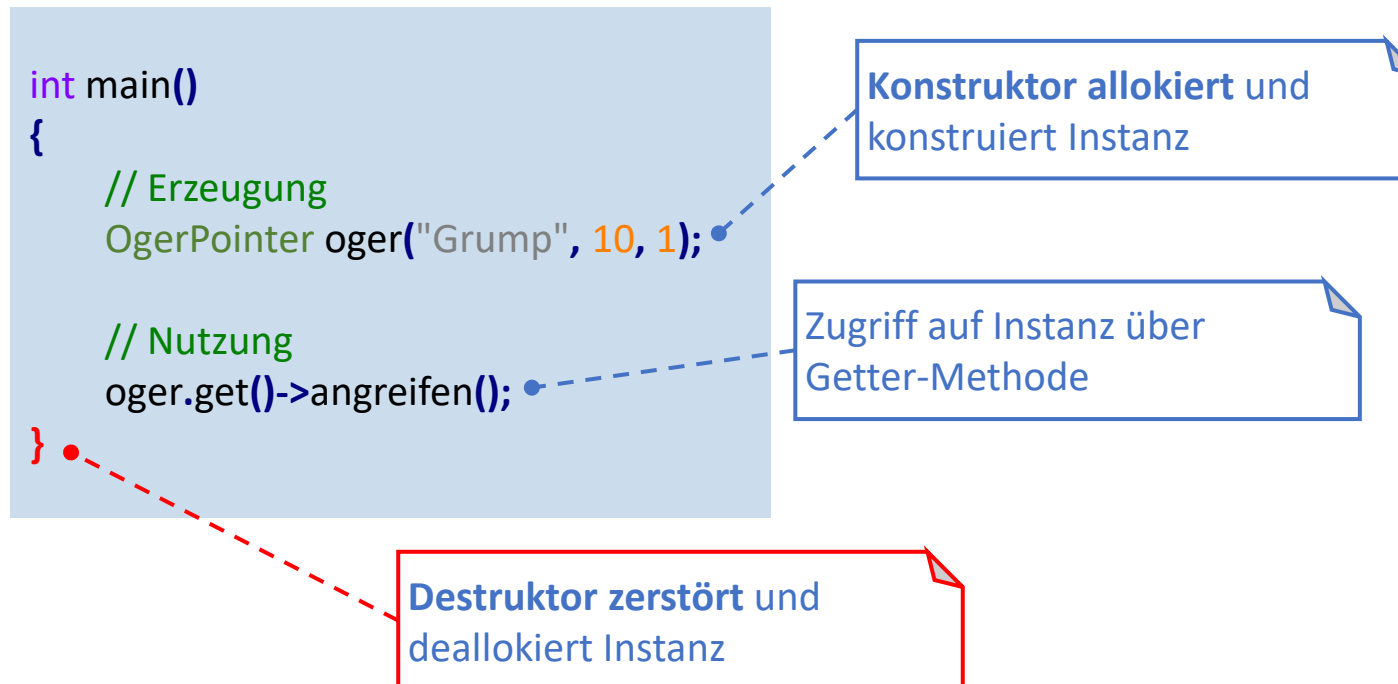
- unique_ptr sind leichtgewichtig und erzeugen **keinen Overhead** im Vergleich zur manuellen Speicherverwaltung von Pointern mittels **new**- und **delete**
 - Es wird lediglich ein Pointer auf das besessene Objekt gespeichert, wie bei anderen Allokationen auch
→ `sizeof(unique_ptr<T>) == sizeof(T*)`
- unique_ptr können **nicht** auf andere Smart-Pointer **kopiert** werden
- unique_ptr können allerdings in anderen Smart-Pointer **verschoben** werden (`std::move()`)



Smart Pointer

unique_ptr – Beispiel

- Erinnern wir uns an das RAII-Objekt **OgerPointer**



- Hier wurde eigentlich bereits ein **unique_ptr** realisiert



Smart Pointer

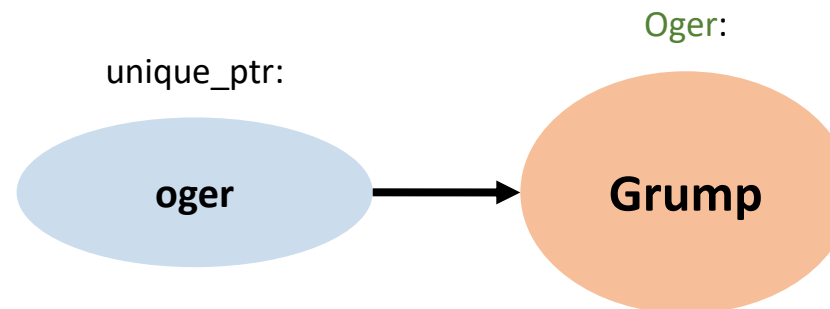
unique_ptr – Beispiel

- Wir können also unsere RAII-Hilfsklasse durch den standardisierten **unique_ptr** ersetzen:

```
#include <memory>

int main()
{
    std::unique_ptr<Oger> oger {
        new Oger("Grump", 10, 1)
    };

    oger.get()->angreifen();
}
```

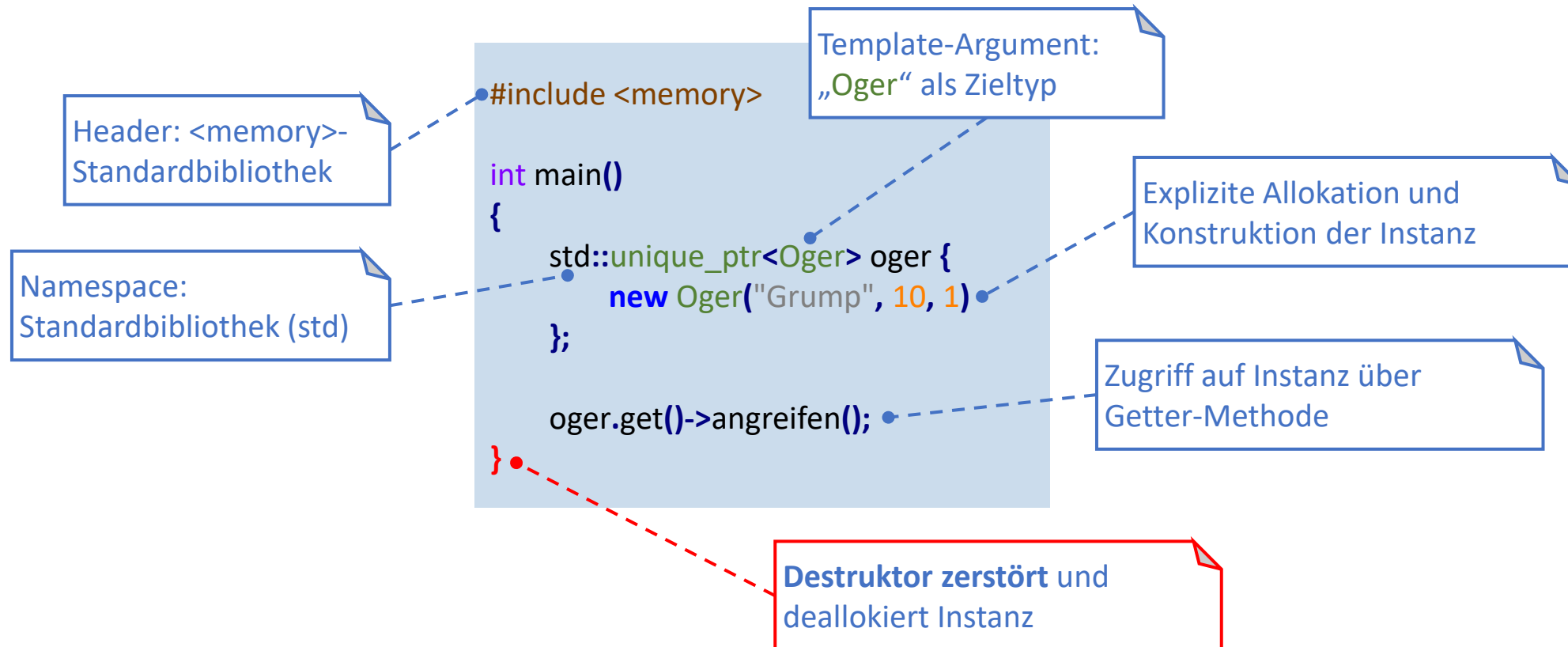


Smart Pointer



unique_ptr – Beispiel

- Der **unique_ptr** übernimmt den Besitz des Oger-Objekts und stellt dessen Freigabe bei Funktionsende sicher – wie bei der **OgerPointer** Hilfsklasse



Smart Pointer



unique_ptr – Beispiel

- Es verbleiben jedoch Unschönheiten:

```
#include <memory>

int main()
{
    std::unique_ptr<Oger> oger {
        new Oger("Grump", 10, 1)
    };

    oger.get()->angreifen();
}
```

Explizite Allokation und
Konstruktion der Instanz

Zugriff auf Instanz über
Getter-Methode

Unschön:

- Expliziter **new**-Aufruf
- Explizite Getter-Methode für Zugriff

Smart Pointer

make()-Funktionen



- Das explizite Allokieren und Konstruieren des Zielobjekts zur Erstellung von Smart-Pointern ist **fehleranfällig**
 - Der übergebene Pointer kann auf ein Objekt zeigen, das sich gar nicht im Heap befindet
→ **undefined behavior** bei Löschung
 - Die Übergabe eines Pointers kann vergessen werden
→ **memory garbage**
→ **undefined behavior**
- Außerdem entspricht die Akquirierung des Zielobjekts außerhalb des Konstruktors (des Smart-Pointers) nicht der RAII-Programmiertechnik!
- Der Standard definiert daher die Funktion **make_unique**:

<memory>	
<pre>template<class T, class... Args> unique_ptr<T> make_unique(Args&&... args)</pre>	• Führt die Allokation und den Konstruktor-Aufruf des Objekts durch • Nimmt nur die Konstruktor-Argumente als Parameter an • Gibt den richtigen Smart-Pointer-Typ zurück

Smart Pointer



unique_ptr – Beispiel

- Die explizite Allokation mit **new** kann durch **make_unique** ersetzt werden:

```
#include <memory>

int main()
{
    std::unique_ptr<Oger> oger {
        std::make_unique<Oger>("Grump", 10, 1);
    };
}
```

Allokiert und konstruiert die
Oger-Instanz

Konstruktor-Argumente

Nun kann auch **auto** als Datentyp für
unique_ptr verwendet werden:

```
#include <memory>

int main()
{
    auto oger {
        std::make_unique<Oger>("Grump", 10, 1);
    };

    oger.get()->angreifen();
}
```

→ **make_unique** sollte der manuellen Erzeugung von Pointern
(mittels **new**) vorgezogen werden

Smart Pointer



Operator-Überladungen

- `unique_ptr` und `shared_ptr` überladen **Operatoren** um direkter – wie normale Pointer – genutzt werden zu können, unter anderem:
- **`operator->`**: Gibt internen Pointer auf das besessene Objekt zurück, damit auf Elemente des Objekts direkt zugegriffen werden kann
- **`operator*`**: Dereferenziert den internen Pointer und gibt damit eine Referenz auf das besessene Objekt zurück
- **`operator bool()`**: Gibt zurück, ob ein besessenes Objekt vorhanden ist oder nicht

```
template <typename T>
class unique_ptr
{
    T* operator->() const
    {
        return m_pointer;
    }

    T& operator*() const
    {
        *m_pointer;
    }

    operator bool() const
    {
        return m_pointer != nullptr;
    }
};
```

Smart Pointer



Operator-Überladungen – Beispiel

- Mit den überladenen Operatoren kann auf die Methode **Oger::angreifen()** direkt zugegriffen werden, ohne den Aufruf der Getter-Methode:

```
#include <memory>

int main()
{
    std::unique_ptr<Oger> oger {
        std::make_unique<Oger>("Grump", 10, 1);
    };

    oger->angreifen();
}
```

Kein Aufruf von **get()** durch
überladenen **->** Operator

Smart Pointer

shared ownership



- Oft muss ein Objekt an mehreren Stellen (Pointern) referenziert werden
- Beispiel: Eine „Wand“-Textur wird geladen und von verschiedenen Komponenten einer Game-Engine genutzt:

```
void gameLoop()
```

```
{
```

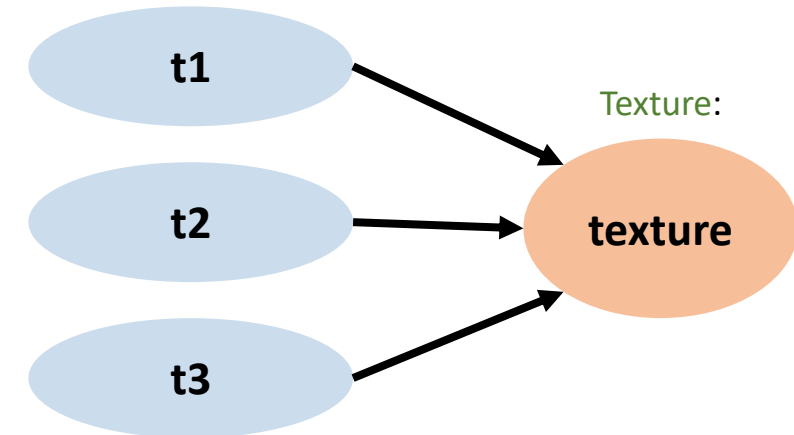
```
Texture* texture { new Texture };  
texture->loadFromFile("wall.png");  
...
```

Textur wird erstellt
und geladen

```
RenderTarget t1("Hauswand", texture);  
RenderTarget t2("Schrankschrank", texture);  
RenderTarget t3("Zimmerwand", texture);  
...
```

Textur wird von drei
rendernden Objekten
genutzt

RenderTargets:



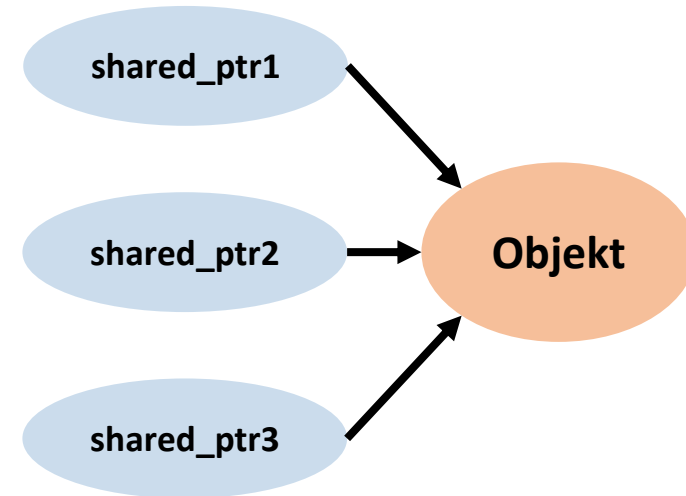
- Hier existiert kein einmaliger Besitz mehr!
- Wer ist für das Löschen der Wand-Textur zuständig wenn keine Komponente mehr rendert?

Smart Pointer

shared_ptr



- **shared_ptr** stellen den **gemeinschaftlichen Besitz** eines Objekts dar
 - Das Objekt, das ein `shared_ptr` *besitzt*, kann von anderen `shared_ptr` geteilt werden
 - Alle `shared_ptr`, die auf das gleiche Objekt zeigen, sind gleichberechtigt
- Ein geteiltes Objekt wird erst zerstört wenn der letzte `shared_ptr` darauf zerstört wurde
- `shared_ptr` können auf andere `shared_ptr` kopiert und verschoben werden
- `shared_ptr` ähneln Objektreferenzen in Programmiersprachen mit automatischer Speicherverwaltung, wie C#, Java oder Python
 - Die Referenzierung wird vom Benutzer verborgen
 - Sie sind jedoch nicht gleich, da der Programmierer immer noch volle Kontrolle über die Zerstörung der Objekte hat (keine Garbage Collection!)



Smart Pointer

shared_ptr



- Wichtigste Funktionen der Klasse (viel analog zu unique_ptr):

shared_ptr<T>	
shared_ptr()	Initialisiert einen leeren shared_ptr
shared_ptr(T*)	Initialisiert einen shared_ptr , der den Besitz des übergebenen Objekts übernimmt
shared_ptr(shared_ptr)	Initialisiert einen shared_ptr , der sich den Besitz des Objekts mit dem übergebenen shared_ptr teilt
~shared_ptr()	Dereferenziert das besessene Objekt und zerstört es, wenn die Referenzanzahl auf null gesunken ist (sofern ein Objekt vorhanden ist)
long use_count()	Gibt die Anzahl von shared_ptr Objekten zurück, die das Objekt besitzen / teilen
T* get()	Gibt einen Pointer auf das besessene Objekt zurück
T* operator->()	Gibt einen Pointer auf das besessene Objekt zurück
T& operator*()	Gibt eine L-Value-Reference auf das besessene Objekt zurück
operator bool()	Gibt zurück ob ein Objekt besessen wird (true) oder nicht (false)
void reset(T* = nullptr)	Ersetzt das besitzende Objekt mit einem anderen. Vorher besessenes Objekt wird dereferenziert und gelöscht, wenn die Referenzanzahl auf null gesunken ist

shared_ptr unterstützen auch die Angabe einer eigenen Löschfunktion für das Objekt – sogenannte „**deleter**“.

Dies führt über den Rahmen der Veranstaltung hinaus und wird nicht näher betrachtet.

Smart Pointer



shared_ptr – make()-Funktion

- Analog zur **make_unique**(...) Funktion definiert der Standard eine **make_shared** Funktion, die ein shared_ptr Objekt auf Basis des Zielobjekts erstellt:

<memory>	
<pre>template< class T, class... Args > shared_ptr<T> make_shared(Args&&... args)</pre>	• Führt die Allokation und den Konstruktor-Aufruf des Objekts durch • Nimmt nur die Konstruktor-Argumente als Parameter an • Gibt den richtigen Smart-Pointer-Typ zurück

Smart Pointer



shared_ptr – Beispiel

- Beispiel: Die „Wand“-Textur wird mit einem shared_ptr als gemeinschaftlich genutztes Objekt der rendernden Komponenten definiert:

```
int main()
{
    std::shared_ptr<Texture> texture { std::make_shared<Texture>() };
    texture->loadFromFile("wall.png");
    ...

    RenderTarget t1("Hauswand", texture);
    RenderTarget t2("Schrankschrank", texture);
    RenderTarget t3("Zimmerwand", texture);
    ...
}
```

Textur als
gemeinschaftliches Objekt

Rendernden Objekte nutzen
Textur-Objekt **gleichberechtigt**

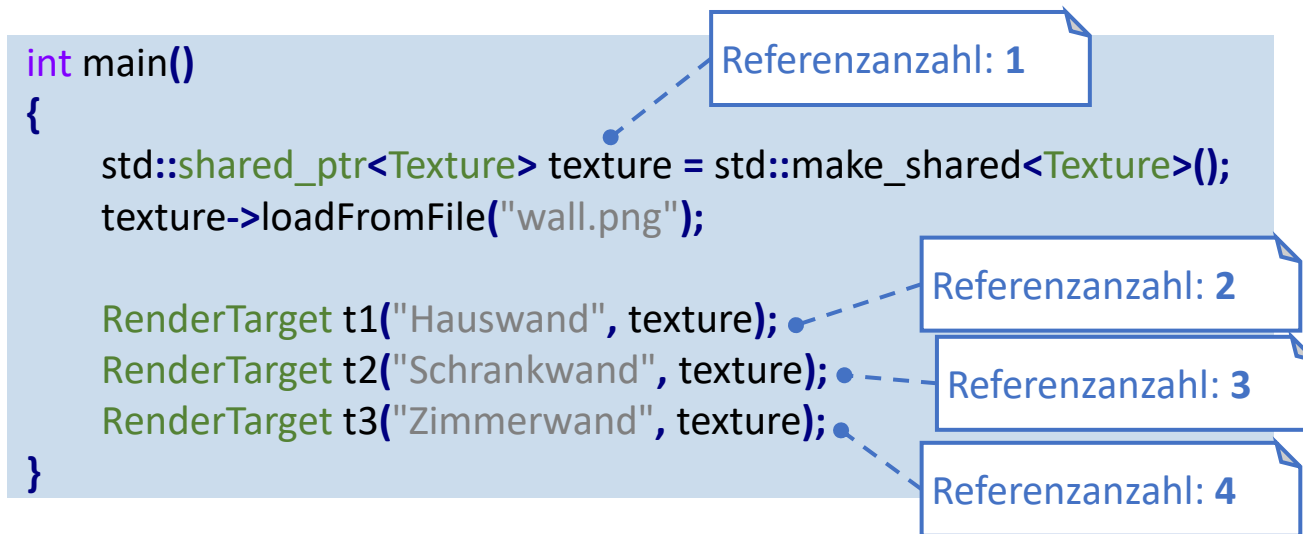
→ Wand-Textur wird gelöscht, wenn die letzte rendernde Komponente gelöscht wurde

Smart Pointer



shared_ptr – Referenzzählung

- Um die **Objektlebenszeit** zu verwalten, zählen shared_ptr die Anzahl der Pointer auf ein Objekt (= **Referenzanzahl** = *reference count*)
 - Mit jeder Erstellung eines Pointers auf das Objekt steigt die Referenzanzahl um eins
 - Mit jeder Zerstörung eines Pointers auf das Objekt sinkt die Referenzanzahl um eins
- Erreicht die Referenzanzahl bei der Zerstörung eines Pointers auf das Objekt null, so wird das Objekt zerstört



Smart Pointer

shared_ptr – Referenzzählung



■ Beispiel: Referenzzählung und Objektzerstörung

```
int main()
{
    std::shared_ptr<Texture> texture1 = std::make_shared<Texture>();

    // Scope 1
    {
        std::shared_ptr<Texture> texture2 = texture1;
        ...

        // Scope 2
        {
            std::shared_ptr<Texture> texture3 = texture1;
            ...
        }
    }
}
```

Referenzanzahl: 1
Pointer und **Objekt** erstellt

Referenzanzahl: 2
Pointer kopiert

Referenzanzahl: 3
Pointer kopiert

Referenzanzahl: 2
Pointer gelöscht

Referenzanzahl: 1
Pointer gelöscht

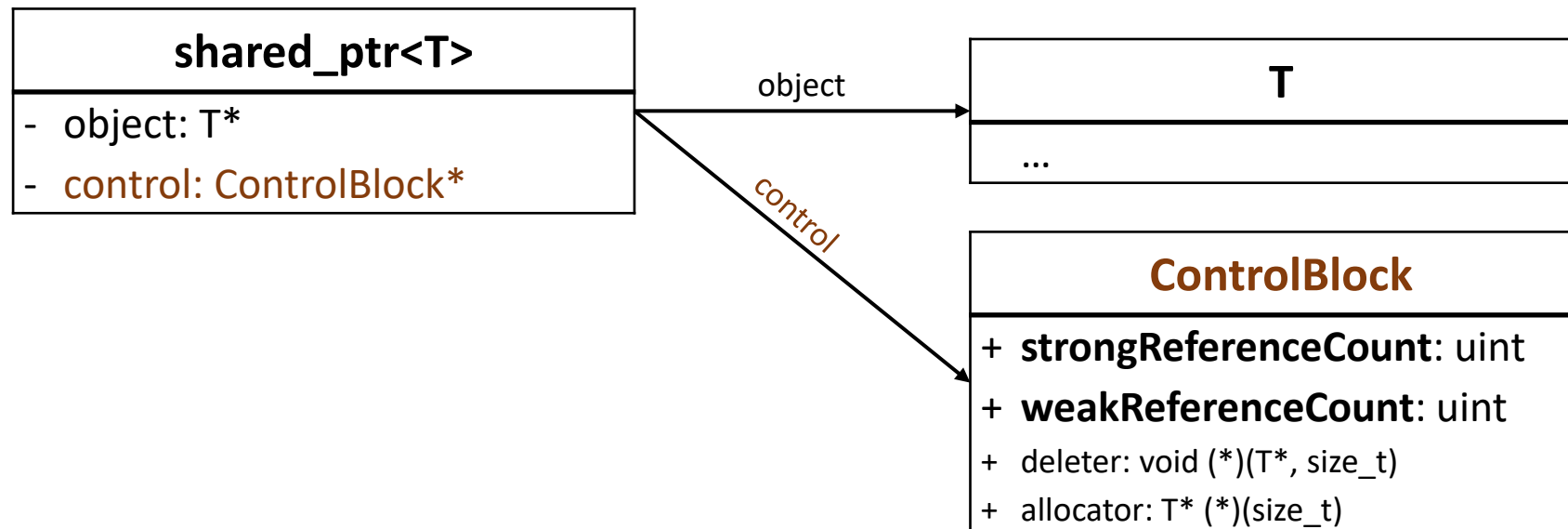
Referenzanzahl: 0
Pointer und **Objekt** gelöscht



Smart Pointer

shared_ptr – Referenzzählung – control block

- Für die Referenzzählung müssen shared_ptr neben dem Objekt eine weitere Variable für die **globale Referenzanzahl** auf dem Objekt verwalten
- Diese und weitere Daten zum geteilten Objekt, befinden sich in einer eigenen Datenstruktur, dem **Control Block**



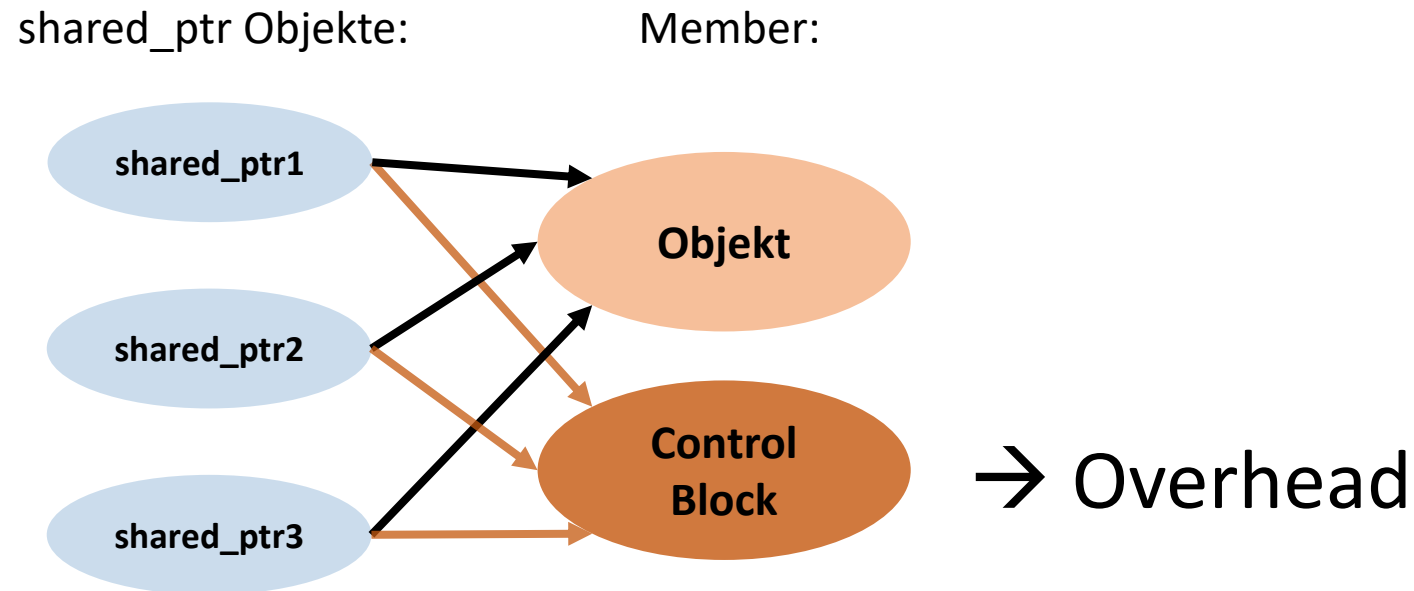
Der control block verwaltet auch explizit angegebene Lösungs- und Allokationsfunktionen („**deleter**“ & „**allocator**“).
Dies führt über den Rahmen der Veranstaltung hinaus und wird nicht näher betrachtet.

Smart Pointer



shared_ptr – Referenzzählung – control block

- Neben dem besessenen bzw. geteilten Objekt kommt also noch die zusätzliche Komponente des Control Blocks hinzu

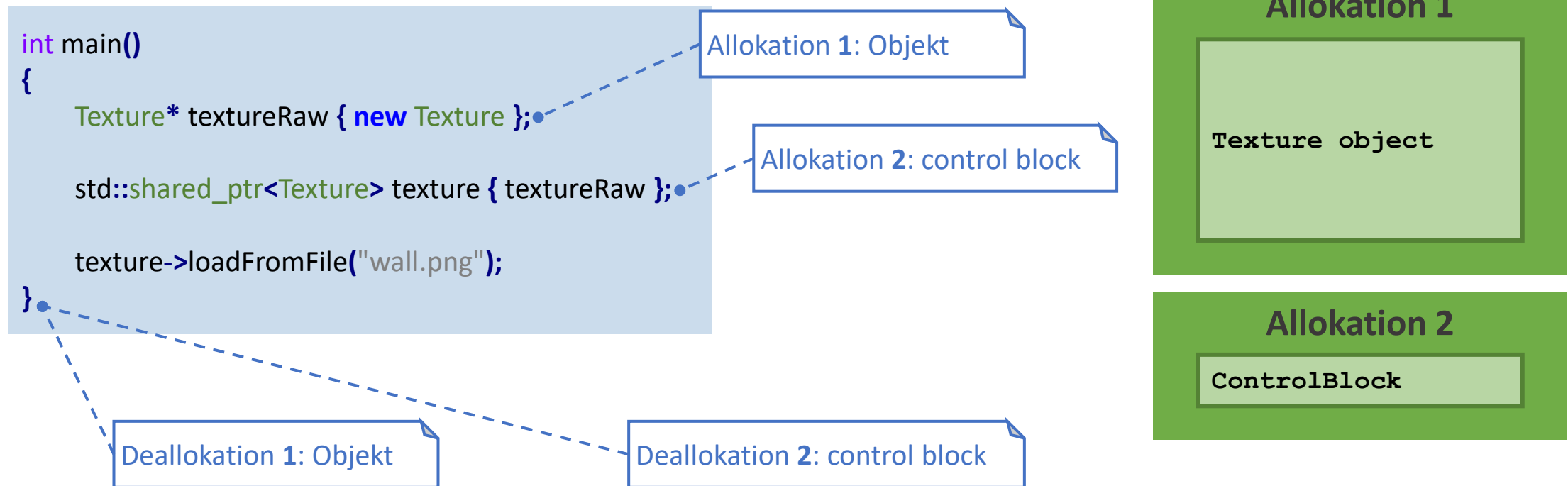


Smart Pointer



shared_ptr – Referenzzählung – control block – Allokation

- Wird ein shared_ptr mit einem Objekt erstellt, das außerhalb des shared_ptr allokiert wurde, muss der Control Block zusätzlich allokiert und wieder freigegeben werden

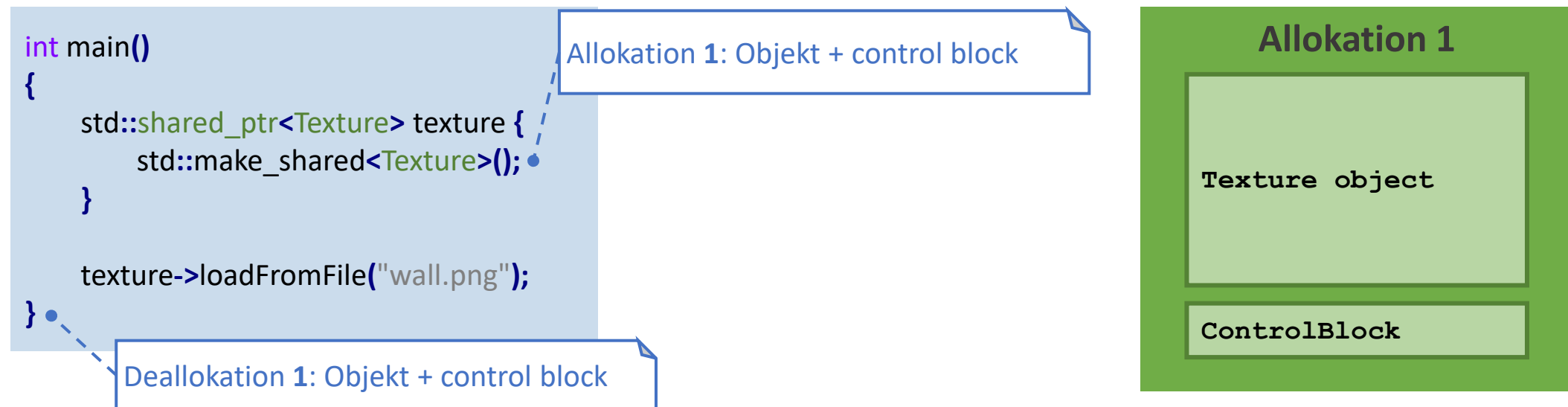


Smart Pointer



shared_ptr – Referenzzählung – control block – Allokation

- Wird ein shared_ptr mit **make_shared(...)** erstellt, kann das Objekt gemeinsam mit dem Control Block allokiert und wieder freigegeben werden



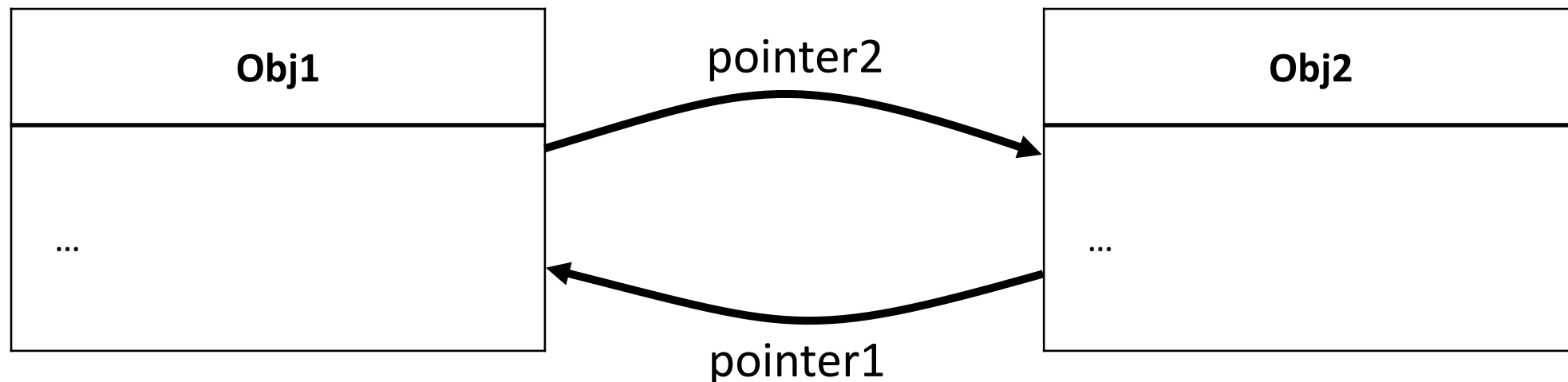
→ make_shared(...) effizienter als separate Objekterstellung

Smart Pointer



shared_ptr – Zyklische Verweise

- Wenn sich zwei oder mehr Objekte über `shared_ptr` gegenseitig referenzieren, kommt es zum Problem der **zyklischen Referenz**
- Dabei verhindern auf sich **gegenseitig zeigende Pointer**, dass die Referenzanzahl dekrementiert und das Objekt dadurch gelöscht wird

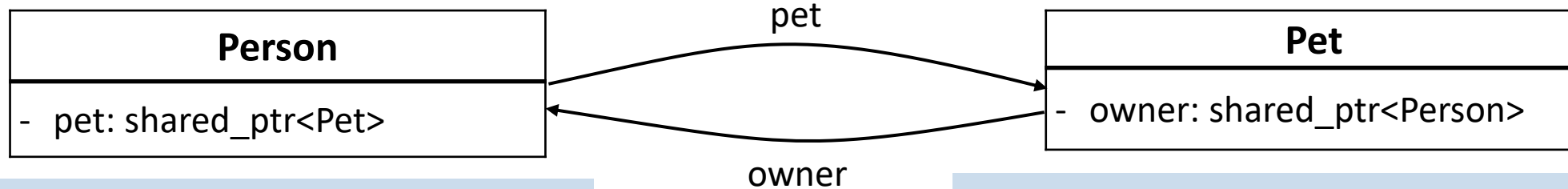


Smart Pointer



shared_ptr – Zyklische Verweise – Beispiel

- Beispiel: zyklische Referenz zwischen einer Person und ihrem Haustier (pet)



```
class Person
{
public:
    void setPet(std::shared_ptr<Pet> pet)
    {
        m_pet = pet;
    }

private:
    std::shared_ptr<Pet> m_pet;
};
```

```
class Pet
{
public:
    void setOwner(std::shared_ptr<Person> owner)
    {
        m_owner = owner;
    }

private:
    std::shared_ptr<Person> m_owner;
};
```

Smart Pointer



shared_ptr – Zyklische Verweise – Beispiel

- Beispiel: zyklische Referenz zwischen einer Person und ihrem Haustier (pet)

```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };

    pet->setOwner(person);
    person->setPet(pet);
}
```

Pet zeigt auf Person (m_owner)

Person zeigt auf Pet (m_pet)

Problem?
Erstmal Schritt für Schritt...



Smart Pointer

shared_ptr – Zyklische Verweise – Beispiel

1. Person und Haustier werden erzeugt

```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };
}
```

Person
Ref: 1

Pet
Ref: 1



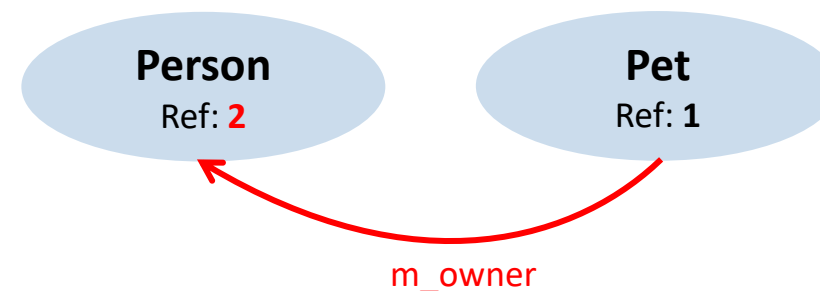
Smart Pointer

shared_ptr – Zyklische Verweise – Beispiel

2. Dem Haustier wird der Besitzer zugewiesen

```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };

    pet->setOwner(person);
}
```





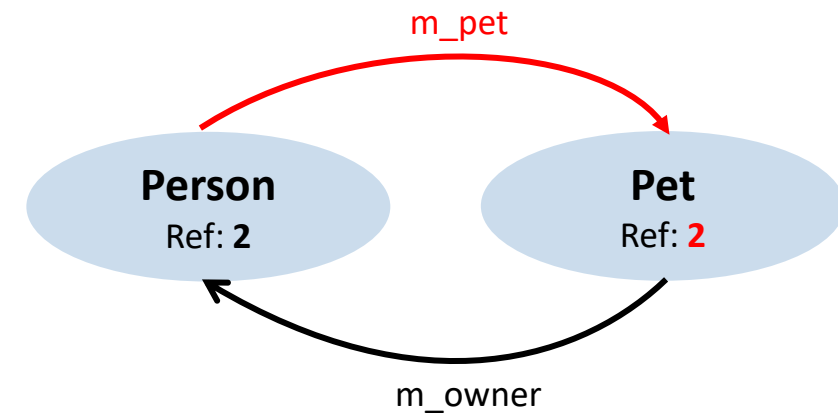
Smart Pointer

shared_ptr – Zyklische Verweise – Beispiel

3. Dem Besitzer wird das Haustier zugewiesen

```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };

    pet->setOwner(person);
    person->setPet(pet);
}
```





Smart Pointer

shared_ptr – Zyklische Verweise – Beispiel

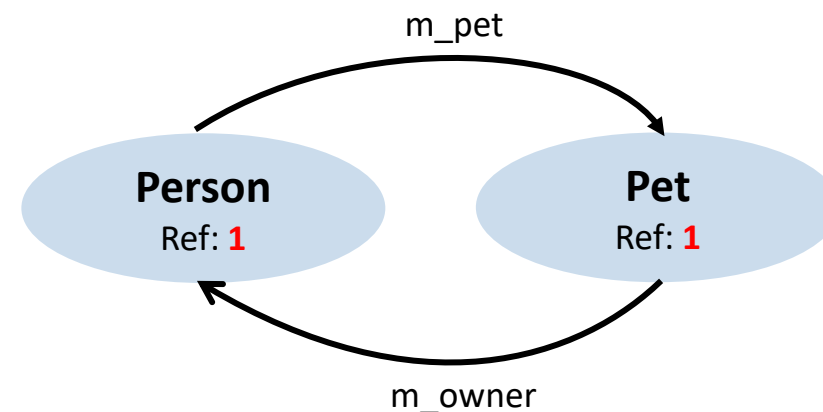
4. Die main()-Funktion endet, wodurch die lokalen Objekte gelöscht werden

→ **person** und **pet** (beides **shared_ptr**) werden gelöscht

```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };

    pet->setOwner(person);
    person->setPet(pet);
}
```

Destruktor-Aufruf der lokalen
Objekte person und pet



Smart Pointer



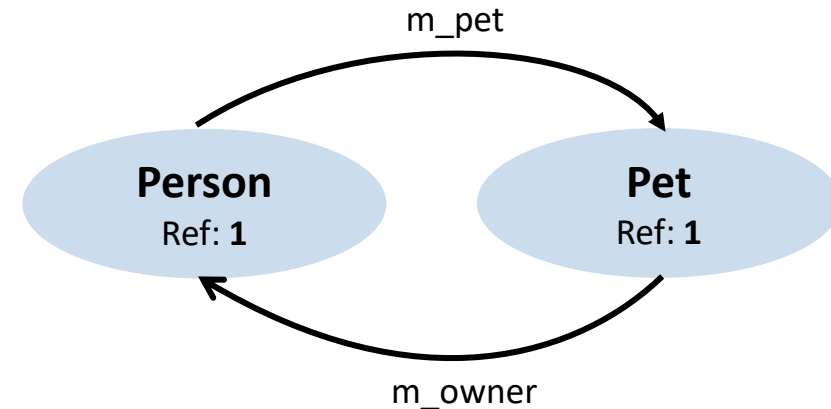
shared_ptr – Zyklische Verweise – Beispiel

⚡ Nach Funktionsende existieren weiter die **internen Verweise** der Objekte „Person“ und „Pet“ **aufeinander!**

```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };

    pet->setOwner(person);
    person->setPet(pet);
}
```

Nach Ende der Funktion:
person.Referenzanzahl: 1
pet.Referenzanzahl: 1



→ Weder die Person noch das Haustier wird gelöscht!
→ Objekt- und Memory-Leak!

Smart Pointer



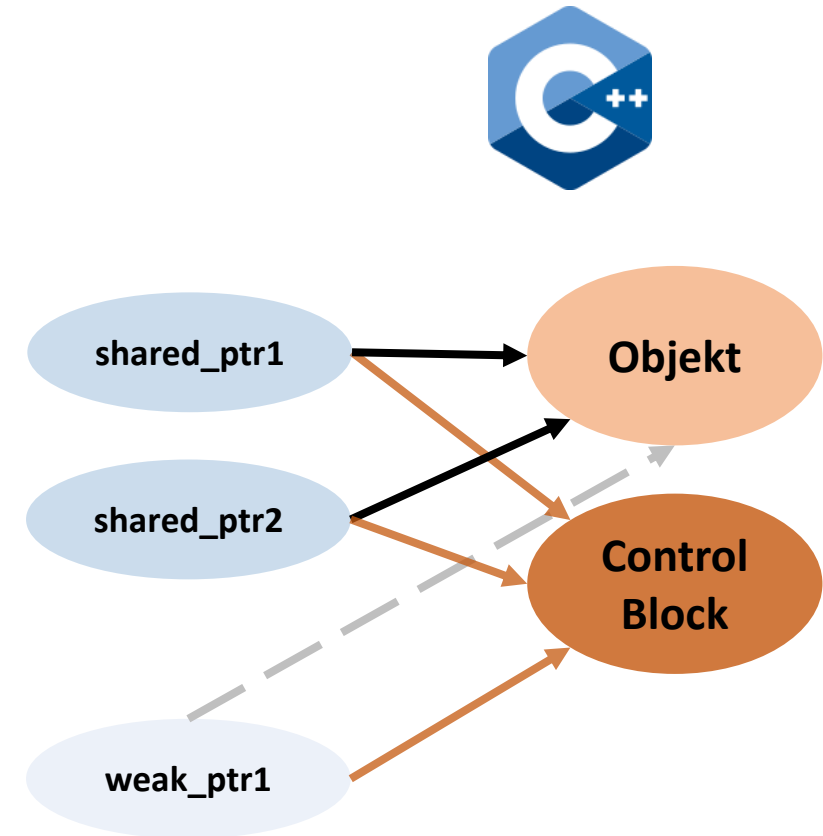
shared_ptr – Zyklische Verweise

- Eine Möglichkeit das Auftreten zyklischer Verweise zu verhindern, ist einen Pointer als **reinen Verweis** zu definieren, der **keine Besitzansprüche** auf das Objekt erhebt
- Welcher der problematischen Pointer das ist, hängt immer von der konkreten Programmarchitektur ab
- Das Design von Eigentumsverhältnissen muss sorgfältig während des Entwurfs einer Software festgelegt werden

Smart Pointer

weak_ptr

- Ein **weak_ptr** („schwacher“ Pointer) verweist auf ein Objekt, das einem shared_ptr gehört
- weak_ptr beeinflussen die Lebenszeit eines Objektes nicht, sie dienen lediglich der **Abfrage des aktuellen Status** eines Objekts
 - Sie beeinflussen jedoch die Lebenszeit des Control Blocks und damit u.U. die gesamte Allokation des shared_ptr
- weak_ptr können als einziger Smart-Pointer-Typ auf **bereits zerstörte** Objekte zeigen = **dangling pointer**





Smart Pointer

weak_ptr – Beispiel

- Beispiel: Einfluss eines weak_ptr auf die Lebenszeit eines Objekts

```
int main()
```

```
{
```

```
    std::weak_ptr<Texture> textureWeak;
```

Referenzanzahl: 0
Pointer ist leer

```
    // Scope 1
```

```
{
```

```
    std::shared_ptr<Texture> textureStrong { std::make_shared<Texture>() };
```

Referenzanzahl: 1
Pointer und Objekt erstellt

```
    textureWeak = textureStrong;
```

```
    ...
```

Referenzanzahl: 1
weak_ptr erhöht die Referenzanzahl nicht

```
    textureWeak;
```

```
}
```

dangling pointer

Pointer zeigt jetzt auf ein **gelöschtes** Texture-Objekt!

Referenzanzahl: 0
Pointer und Objekt gelöscht

Smart Pointer



weak_ptr – Zyklische Verweise – Beispiel

- Das Problem der zyklischen Referenz im Beispiel zwischen der Person und ihrem Haustier kann gelöst werden, indem einer der beiden **shared_ptr-Member mit einem weak_ptr-Member ersetzt** wird
 - Z.B. der Zeiger auf den Besitzer (m_owner) im Haustier als weak_ptr:

```
class Pet
{
public:
    void setOwner(std::shared_ptr<Person> owner)
    {
        m_owner = owner;
    }

private:
    std::weak_ptr<Person> m_owner;
};
```

weak_ptr beeinflusst die
Referenzanzahl auf Person nicht



Smart Pointer

weak_ptr – Zyklische Verweise – Beispiel

1. Person und Haustier werden erzeugt

```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };
}
```



unverändert



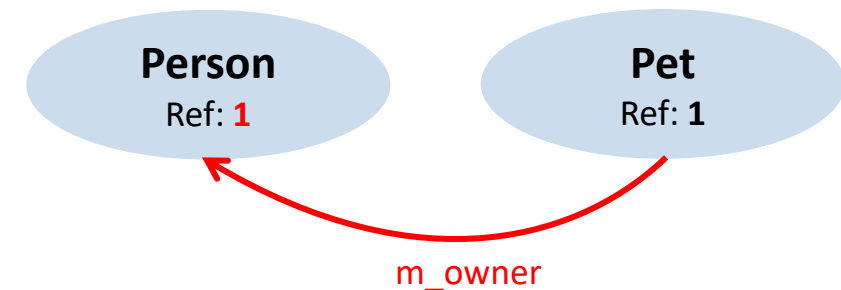
Smart Pointer

weak_ptr – Zyklische Verweise – Beispiel

2. Dem Haustier wird der Besitzer zugewiesen

```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };

    pet->setOwner(person);
}
```



neu: Referenzanzahl auf Person steigt nicht!
Da **m_owner** jetzt weak_ptr statt shared_ptr ist

Smart Pointer

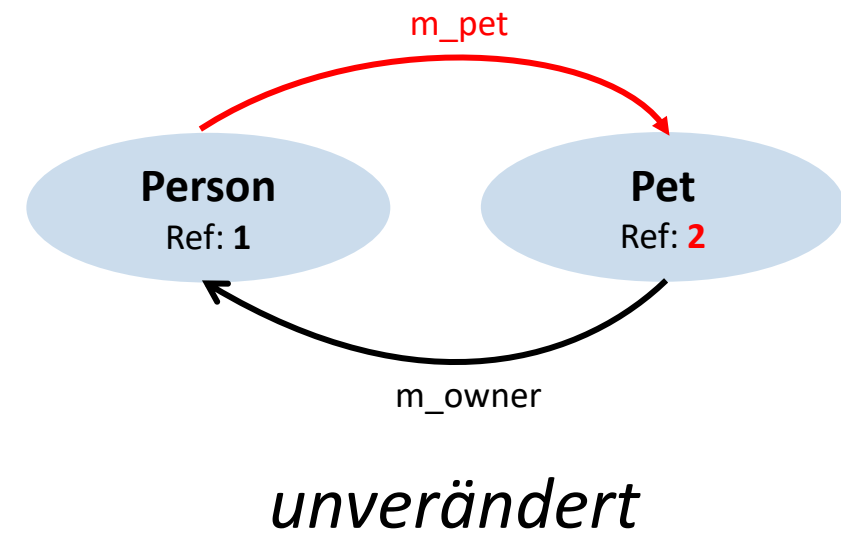
weak_ptr – Zyklische Verweise – Beispiel

3. Dem Besitzer wird das Haustier zugewiesen



```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };

    pet->setOwner(person);
    person->setPet(pet);
}
```





Smart Pointer

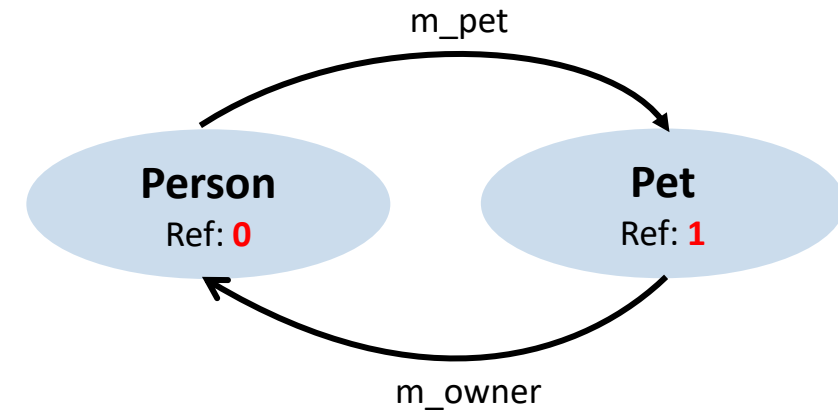
weak_ptr – Zyklische Verweise – Beispiel

4. Die main()-Funktion endet, wodurch die lokalen Objekte gelöscht werden
→ **person** und **pet** (beides **shared_ptr**) werden gelöscht

```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };

    pet->setOwner(person);
    person->setPet(pet);
}
```

Destruktor-Aufruf der lokalen
Objekte person und pet



unverändert

Smart Pointer



weak_ptr – Zyklische Verweise – Beispiel

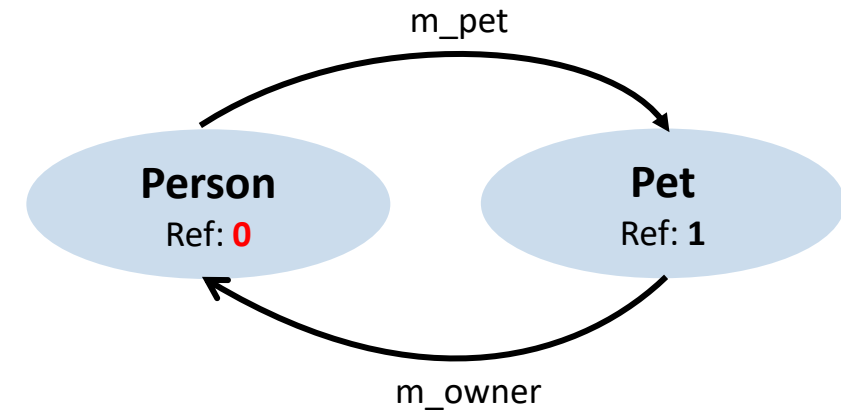
5. Die Referenzanzahl auf der Person fällt auf **0**, da die Zuweisung von **m_owner** in Pet diese als weak_ptr nicht erhöht hat

→ **person**-Objekt wird **vollständig** gelöscht!

```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };

    pet->setOwner(person);
    person->setPet(pet);
}
```

Nach Ende der Funktion:
person.Referenzanzahl: **0**
pet.Referenzanzahl: **1**



neu!

Smart Pointer



weak_ptr – Zyklische Verweise – Beispiel

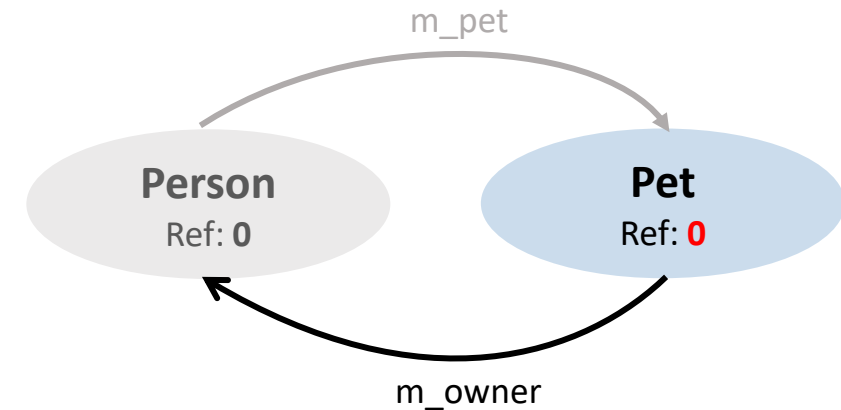
6. Bei der Zerstörung des **person**-Objekts wird der **m_pet** Pointer zerstört, wodurch die Referenzanzahl auf dem **pet**-Objekt auf **0** fällt

→ **pet**-Objekt wird **vollständig** gelöscht!

```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };

    pet->setOwner(person);
    person->setPet(pet);
}
```

Nach Ende der Funktion:
person.Referenzanzahl: 0
pet.Referenzanzahl: 0



neu!

Smart Pointer



weak_ptr – Zyklische Verweise – Beispiel

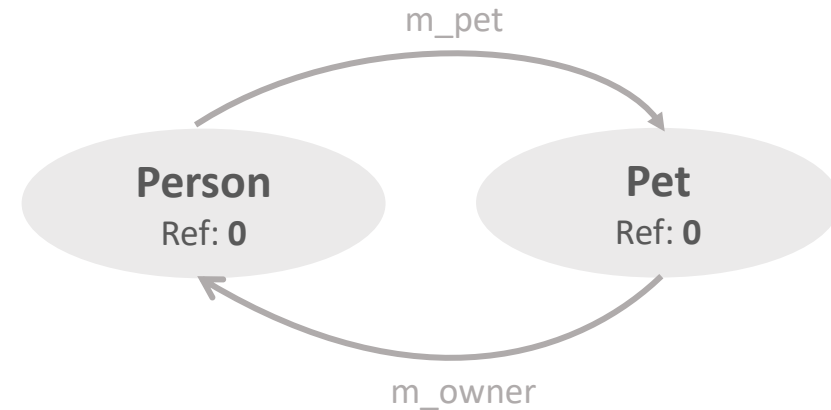
- ✓ Durch die schwache Referenz im Pet auf die Person, wurde nach Funktionsende das „Person“ Objekt gelöscht, wodurch wiederum das „Pet“ Objekt gelöscht wurde

```
int main()
{
    std::shared_ptr<Person> person { std::make_shared<Person>() };
    std::shared_ptr<Pet> pet { std::make_shared<Pet>() };

    pet->setOwner(person);
    person->setPet(pet);
}
```

Referenzanzahl auf
Person stieg nicht

Nach Ende der Funktion:
person.Referenzanzahl: 0
→ pet.Referenzanzahl: 0



→ Person und Haustier
wurden korrekt gelöscht

Smart Pointer



weak_ptr – Nutzung

■ Wichtigste Funktionen der Klasse:

weak_ptr<T>	
weak_ptr()	Initialisiert einen leeren weak_ptr
weak_ptr(shared_ptr)	Initialisiert einen weak_ptr, der auf den übergebenen shared_ptr und damit indirekt auf das besessene Objekt von diesem verweist
~weak_ptr()	Zerstört die Referenz auf den shared_ptr, führt keine Objektzerstörung aus!
long use_count()	Gibt die Anzahl von shared_ptr Objekten zurück, die das indirekt referenzierte Objekt besitzen
bool expired()	Gibt zurück ob das indirekt referenzierte Objekt bereits gelöscht wurde (true) oder nicht (false)
shared_ptr lock()	Erstellt einen neuen vollwertigen shared_ptr auf das indirekt referenzierte Objekt
void reset()	Gibt die schwache Referenz auf das Objekt frei, führt keine Objektzerstörung aus!

Smart Pointer



weak_ptr – Nutzung

- Mit der Methode **lock()** kann ein vollwertiger `shared_ptr` auf das indirekt verwiesene Objekt erhalten werden
 - Existiert das Objekt (Referenzanzahl > 0), gibt die Methode einen **neuen `shared_ptr`** auf das Objekt zurück (→ Referenzanzahl += 1)
 - Wurde das Objekt gelöscht (Referenzanzahl = 0), gibt die Methode einen **`nullptr`** zurück
- Die Abfrage ob ein Objekt existiert oder nicht, kann also über einen einfachen **Nullcheck** erfolgen
- Alternativ kann über die Methode **`expired()`** abgefragt werden, ob das Objekt, auf das der `weak_ptr` verweist, bereits gelöscht wurde
- Bei `weak_ptr` und `shared_ptr` kann außerdem über die Methode **`use_count()`** abgefragt werden, wie viele `shared_ptr` (= strong references) auf dem Objekt gerade existieren

Smart Pointer



weak_ptr – Nutzung – Beispiel

- Beispiel: Zugriff auf den Besitzer vom Haustier über weak_ptr

```
class Pet
{
public:
    void setOwner(std::shared_ptr<Person> owner);
    void greetOwner();

private:
    std::weak_ptr<Person> m_owner;
};
```

```
void Pet::greetOwner()
{
    std::shared_ptr<Person> owner { m_owner.lock() };
    if (owner != nullptr)
    {
        owner->notifyGreet();
    }
    else
    {
        std::cout << "Error: I have no owner :(";
    }
}
```

lock() gibt shared_ptr auf Objekt zurück

Objekt existiert und kann verwendet werden

Objekt existiert nicht (mehr)

Smart Pointer



weak_ptr – Nutzung – Beispiel

- Neben der Vermeidung zyklischer Referenzen eignen sich weak_ptr auch gut für das „Caching“ bzw. „Lazy Loading“ von Objekten

TextureFactory	
-	m_textureWall: weak_ptr<Texture>
+	getOrCreateWallTexture(): shared_ptr<Texture>

TextureFactory beeinflusst die Lebenszeit der erstellten Textur nicht

Erstellen und Laden der Textur, wenn sie nicht (mehr) vorhanden ist

Speichern eines reinen Verweises auf die Textur für spätere Aufrufe

```
std::shared_ptr<Texture> TextureFactory::getOrCreateWallTexture()
{
    std::shared_ptr<Texture> textureWall { m_textureWall.lock() };

    if (textureWall == nullptr)
    {
        textureWall = std::make_shared<Texture>();
        textureWall->loadFromFile("wall.png");

        m_textureWall = textureWall;
    }

    return textureWall;
}
```

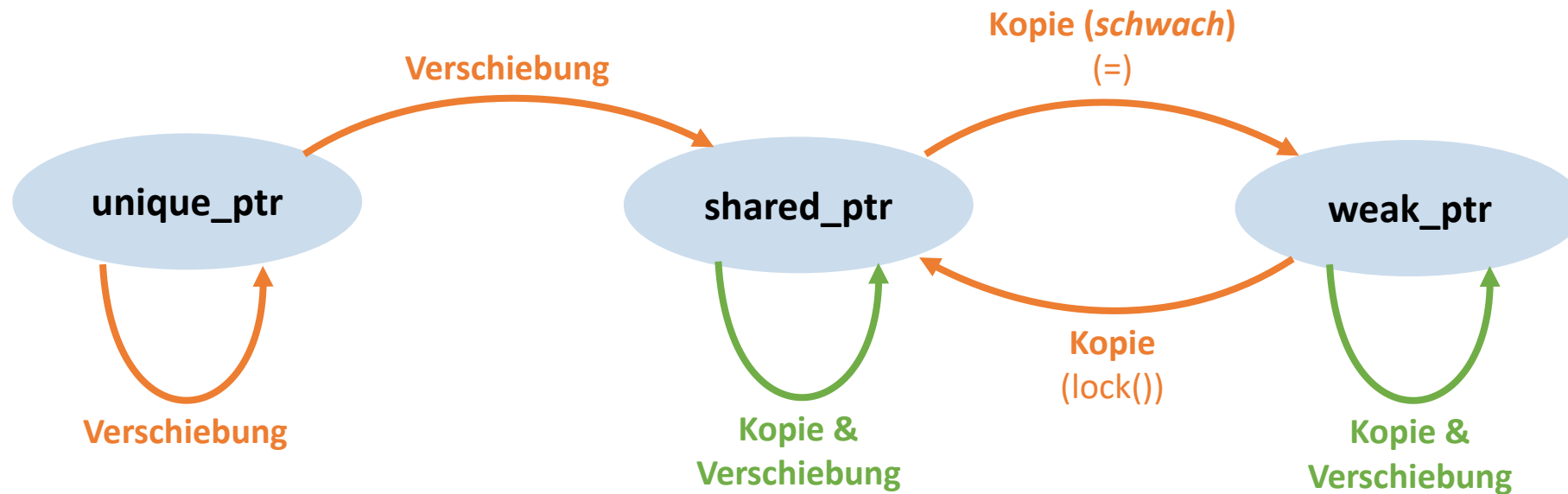
Rückgabe der gespeicherten Textur oder der neu-erstellten, wenn keine Textur vorhanden war
→ Caching

Smart Pointer



Konvertierbarkeit

- **Zuweisungen** unter Smart-Pointern bilden **Änderungen an den Eigentumsverhältnissen** von Objekten ab
- Dabei hängt die Konvertierbarkeit von der Art der drei Pointertypen ab:



Nicht-angegebene Konversionen sind nicht zulässig!

Smart Pointer

Konvertierbarkeit



- Ausgedrückt als Tabelle:
 - Pfeile (→) stellen die Zuweisung (=) dar

Zuweisung	Konvertierbarkeit
unique_ptr → unique_ptr	nur Verschiebungen
shared_ptr → shared_ptr	Kopien und Verschiebungen
weak_ptr → weak_ptr	Kopien und Verschiebungen
unique_ptr → shared_ptr	Konversion durch Verschiebung
unique_ptr → weak_ptr	nicht zulässig!
shared_ptr → unique_ptr	nicht zulässig!
weak_ptr → unique_ptr	nicht zulässig!
shared_ptr → weak_ptr	Schwache Referenzierung
weak_ptr → shared_ptr	Erhalt über lock()

keine Kopien zulässig!

keine Verschiebung zulässig!

Smart Pointer



Konvertierbarkeit – Beispiel

- Beispiel: Konvertierung von einzigartigem zu gemeinschaftlichem Besitz eines Objekts

```
int main()
{
    std::unique_ptr<Texture> texture { std::make_unique<Texture>() };
    texture->loadFromFile("wall.png");
    ...

    std::shared_ptr<Texture> textureShared { std::move(texture) };
    ...
}
```

Textur wird als einzigartiges (unique) Objekt erstellt

Einzigartiges Textur-Objekt wird mittels **Verschiebung** zu gemeinschaftlich genutztem Objekt konvertiert

→ `unique_ptr` wurde zu `shared_ptr` konvertiert
(*Texture-Objekt wurde nicht kopiert oder verschoben, nur Smart-Pointer darauf*)

Smart Pointer



Konvertierbarkeit – Casting

- Beim **Casting** von Smart-Pointern ist meist das **Casten des Zieltyps** erwünscht, nicht das Casten des Smart-Pointers selbst
 - Smart-Pointer selbst sollten nie gecastet werden, da das zu schwerwiegenden Problemen der zugrundeliegenden Datenstrukturen führt
- Der Standard definiert daher Methoden, mit denen die C++-Casts auf Smart-Pointer angewendet werden können
 - Diese Methoden erzeugen Compilerfehler, wenn der Cast unzulässig ist

Smart-Pointer-Cast	Äquivalenter Raw-Pointer-Cast
<code>std::static_pointer_cast<TZiel>(p);</code>	<code>static_cast<TZiel>(p);</code>
<code>std::dynamic_pointer_cast<TZiel>(p);</code>	<code>dynamic_cast<TZiel>(p);</code>
<code>std::const_pointer_cast<TZiel>(p);</code>	<code>const_cast<TZiel>(p);</code>
<code>std::reinterpret_pointer_cast<TZiel>(p);</code>	<code>reinterpret_cast<TZiel>(p);</code>

Smart Pointer



Konvertierbarkeit – Casting – Beispiel

- Beispiel: Konstantes Objekt zu veränderbarem Objekt casten unter Verwendung von Smart-Pointern:

```
int main()
{
    std::shared_ptr<const Texture> texture { std::make_shared<const Texture>() };
    ...

    std::shared_ptr<Texture> textureMut { std::const_pointer_cast<Texture>(texture) };
    textureMut->loadFromFile("wall.png");
    ...
}
```

Nicht-veränderbares Objekt

Smart-Pointer-Cast erstellt einen neuen `shared_ptr` für die `textureMut` Variable (Könnte auch verschoben werden)

→ Kopie wurde korrekt erstellt

Smart Pointer



Zusammenfassung

- **unique_ptr** sind *leichtgewichtige* Pointer für Objekte, die nicht gemeinschaftlich genutzt werden
- **shared_ptr** erlauben die sichere gemeinschaftliche Nutzung von Objekten
 - Sie sind „**teurer**“ in der Nutzung durch den **Overhead** der Referenzzählung und des **control blocks**
- **weak_ptr** erlauben den Verweis, ohne Besitzansprüche, auch auf gelöschte Objekte
 - Sie sind sinnvoll bei der Lösung des Problems von **zyklischen Verweisen**...
 - und bei Architekturen, bei denen Objekte referenziert, aber nicht besessen werden sollen (z.B. Caching)
- Smart-Pointer sichern die **korrekte Zerstörung** von Objekten durch die RAII-Programmiertechnik
- Smart-Pointer sollten mit den **make_(-)-Methoden** erstellt, den **..._pointer_cast(p)-Methoden** gecastet, der **std::move(-)-Methode** verschoben und sonst trivial zugewiesen werden
- Die Auswahl der richtigen Smart-Pointer-Typen hängt von dem **Entwurf des Gesamtsystems** ab

Smart Pointer

Verständnisfragen

PIN: 618987

<https://poll.hs-kempten.de/poll/>



- **Welche der folgenden Pointer-Typen arbeiten mit Referenzzählung?**
 - a) rohe Zeiger
 - b) `unique_ptr`
 - c) `shared_ptr`
 - d) `weak_ptr`
- **Was ist der Hauptvorteil von Smart-Pointern gegenüber rohen Zeigern?**
 - a) Sie sind schneller als rohe Zeiger.
 - b) Sie verwalten automatisch die Lebenszeit und Freigabe eines Objekts.
 - c) Sie erlauben zyklische Referenzen zwischen Objekten.
 - d) Sie vereinfachen die dynamische Objekterstellung.
- **Welchen Hauptzweck erfüllt `weak_ptr`?**
 - a) Kopien von `shared_ptr` zu ermöglichen.
 - b) Einen Kompromiss zwischen `unique_ptr` und `shared_ptr` zu schaffen.
 - c) Referenzierung eines Objekts, ohne dessen Lebenszeit zu beeinflussen.
 - d) Abfrage der Anzahl von Referenzen auf einem Objekt.

Smart Pointer

Verständnisfragen

PIN: 618987

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Der Begriff „strong reference“ bei einem Smart-Pointer bedeutet, dass der Pointer für die Löschung des Objekts verantwortlich ist.
- Ein „dangling pointer“ ist ein Zeiger auf ein bereits gelöscht Objekt.
- Ein `std::unique_ptr` kann kopiert werden.
- `std::unique_ptr` verbrauchen genau so viel Speicher wie normale Pointer (kein Overhead).
- Ein durch `std::shared_ptr` geteiltes Objekt wird dann gelöscht, wenn mindestens ein `shared_ptr` darauf gelöscht wird.
- Ein `std::weak_ptr` besitzt ein Objekt, wie es auch beim `std::shared_ptr` der Fall ist.
- Über einen `std::weak_ptr` kann die Anzahl der Smart-Pointer erhalten werden, die das besessene Objekt referenzieren.
- Smart-Pointer können problemlos mit C++-Casts, wie `static_cast`, umgewandelt werden.



Fazit / Abschluss

- Die bisher vorgestellten Sprachkonzepte und Datenstrukturen erlauben die Verwaltung von Ressourcen auf einem **hohen Abstraktionsniveau**
 - Konstruktoren und Zuweisungsoperatoren
 - Kopier- und Verschiebungsoperationen
 - RAI
 - Smart-Pointer
 - Container (kommendes Kapitel)
- In modernem C++ wird der **manuelle Aufruf** von Funktionen zur Ressourcenverwaltung **überflüssig** ...
 - wie der new- und delete-Operatoren (bzw. malloc()- und free()-Funktionen) oder auch ...
 - Thread-Handles oder andere Betriebssystemressourcen
- Stattdessen wird der Fokus auf die Definition von **Objekten und Eigentumsverhältnissen** gelegt

Moderne Ressourcenverwaltung

Beispiel



Modernes C++

```
int main()
{
    auto textureOrig { std::make_shared<Texture>() };
    textureOrig->loadFromFile("wall.png");

    auto textureShallowCopy { textureOrig };

    auto textureDeepCopy { std::make_shared<Texture>(*textureOrig) };
}
```

Traditionelles C++

```
int main()
{
    Texture* textureOrig = new Texture;
    textureOrig->loadFromFile("wall.png");

    // keine Referenzzählung!
    Texture* textureShallowCopy = textureOrig;

    // umständlich
    Texture* textureDeepCopy = new Texture;
    textureDeepCopy->copyFrom(textureOrig);

    // kann leicht vergessen werden:
    delete textureDeepCopy;
    // Fehler!
    delete textureShallowCopy;
    // kann leicht vergessen werden:
    // double free -> heap corruption
    delete textureOrig;
}
```